一种新型通用处理器架构

祝林 翟冬青

摘 要 通用处理器架构经历了复杂指令集计算机（Complex Instruction Set Computer，CISC）和精简指令集计算机（Reduced Instruction Set Computer，RISC）两个阶段。但发展到现在，渐渐暴露出一些缺陷，它们既不能充分利用当前芯片的制造技术，也不能满足用户对大量数据处理速度的需求。为此，我们提出了一种新的通用处理器架构：祝氏共享计算机 (Zhu's Sharing Set Computer，ZSSC)：共享功能模块的计算器。功能模块实现了复杂运算和系统服务等功能。

复杂运算包含使用频率低的运算和需要占用非常大芯片面积才能实现的运算。

系统服务包含硬件资源的管理和数据处理等服务。

RISC 是根据“80%的工作是由 20%的指令完成”（“80% of the work is done by 20% of the instructions”）的统计结果而提出的一种处理器架构。它的设计策略（processor design strategy）只关注了比较常用的简单指令，而忽略了其他指令。相较于 RISC 架构，ZSSC 架构完善了“80/20 法则”在处理器领域的设计哲学（design philosophy）：处理器以共享的方式分担功能模块的硬件实现成本。以这种方式，将简单操作、复杂运算、系统服务等操作和谐统一在同一个处理器架构下，既完成了资源的极大化使用，又能完成大量数据高速处理。本文将在 ZSSC 架构下提出并讨论一种处理器模型：简单祝氏处理器 (basic Zhu's Computer Processor，BZCP) 。

关键词：祝氏共享计算机(ZSSC)；简单祝氏处理器(BZCP)；祝氏指令集架构(ZISA)

1 引言

处理器组成元件经历了电子管，晶体管，小规模集成电路（Small-scale integration，SSI），大规模集成电路（Large-scale integration，LSI）和甚大规模集成电路（Very-Large-scale integration，VLSI）等几个时代[1]。早期的处理器能使用的元件非常少，各种计算机的机器指令都很简单，且各不相同。针对指令不兼容的情况，IBM 于 1964 年发布了 System 360，从此，机器指令集从处理器硬件中独立出来，称为指令集架构（Instruction Set Architecture，ISA）[2]。随着芯片制造工艺的提升，处理器的指令集也越来越复杂。

IBM 研究中心的 John Cocke 发现，计算机中约 20%的指令承担了 80%的工作。之后，1979 年，D. A. Pattersonn 等人提出了 RISC 的思想[1]。在 RISC 提出之后，人们才将之前的指令集称为复杂指令集（CISC）。

RISC 架构的出现不仅诞生了很多优秀的处理器：MIPS、Alpha、PowerPC、SPARC、ARM 等，也开发出很多提升处理器性能的技术：流水线（Pipeline）技术，超标量（Superscalar）技术，超流水线（Superpipeline）技术，乱序执行（Out-of-order Execution）技术，分支预测（Branch Prediction）技术等。推动处理器性能快速提升的因素除了指令集架构和各种技术外，芯片制造工艺的进步也功不可没。

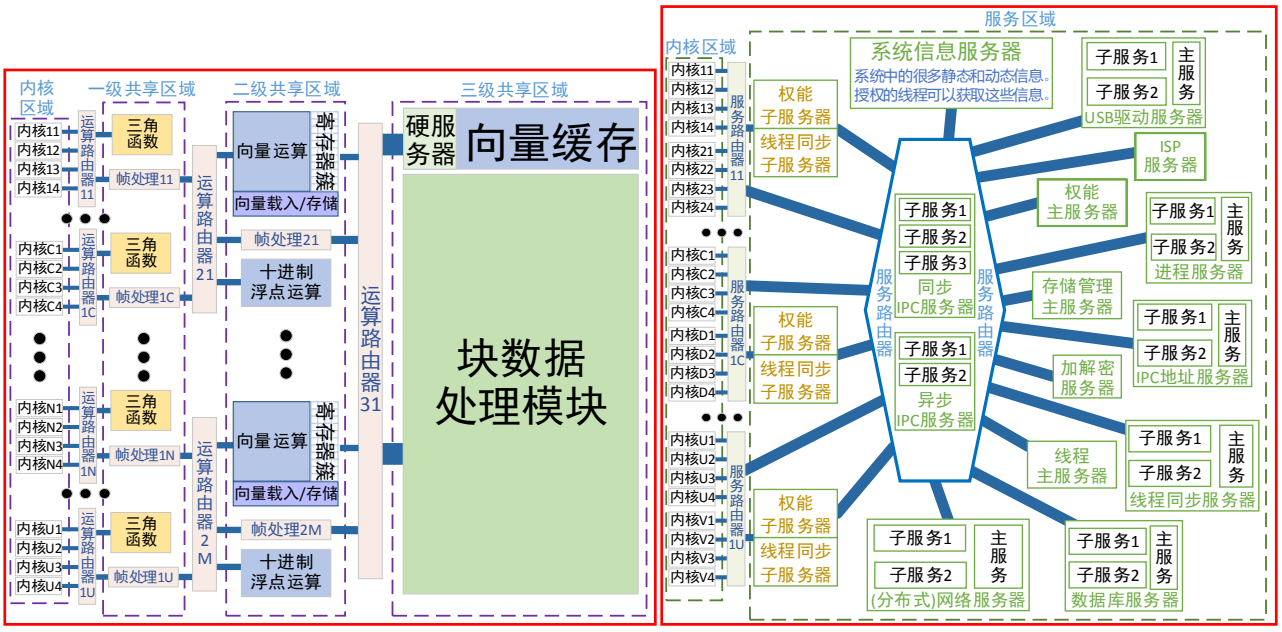
几十年时间里，通用处理器从无到有，从单核到多核，从慢到快飞速发展。但是，当前的处理器架构师和设计师很可能无法维持通用处理器中的显著性能改进[3]。另一方面，图形处理器（Graphics Processing Unit，GPU）携带着 CUDA（Compute Unified Device Architecture）强势登场，其计算能力让人惊叹不已。随后，各种针对应用领域做优化的处理器架构（Domain Specific Architecture，DSA）[3]纷纷登场：google 的张量处理器（Tensor Processing Unit，TPU），寒武纪的神经网络处理器（Neural network Processing Unit，NPU）等。新的通用处理器架构需要能提供媲美这些 DSA 的运算能力，而 ZSSC 恰好可以。

本文在 ZSSC 架构下提出一种简单的通用处理器模型 BZCP。通过对 BZCP 处理器整体框架和局部模块的描述，展示了同一个架构下，精简指令，复杂指令和服务指令各自的执行过程，诠释了“80/20 法则”（“80/20 rule”）在处理器设计中的指导作用。

BZCP 处理器的是一种众核处理器，它不苛求单个线程的运行速度，而是追求对片上资源的合理分配，使其在相同晶体管数量下，完成同一组任务时使用的时间最短。因为 BZCP 处理器包含多个精简处理器，所以多任务并行运行是其内禀属性。

BZCP 处理器的指令是复杂指令（第 4 章中介绍），这是 ZSSC 架构的特点。可以看到，处理器的指令从最初的简单到复杂（CISC）再到简单（RISC），最后又回到复杂（ZSSC），呈现一种螺旋式上升的发展过程。

2 BZCP 处理器的整体结构



(a)处理器的运算系统　　(b)处理器的服务系统

图 2.1　BZCP 处理器的整体结构（不包含存储系统）

BZCP 处理器（图 2.1）由运算系统和服务系统两部分组成。其中，图 2.1（a）处理器的运算系统由内核区域与运算区域构成；图 2.1（b）处理器的服务系统由内核区域与服务器区域构成。

内核区域包含处理器内所有内核模块（后面简称为：内核）。内核的主要功能是执行系统和用户的程序。所有的用户程序只能在内核区域中运行。内核同时具有运算功能和服务功能。

运算区域包含所有共享的复杂运算模块。复杂运算模块的主要功能是运算处理各种数据。按照运算的类型，复杂运算模块分为复杂标量运算模块，向量处理模块，块数据处理模块。内核通过运算帧和信息帧与各种运算模块交互。

服务器区域由各种微型硬件服务器（后面简称为：硬服务器，是包含特定外设，运行特定程序的微处理器）组成，是服务系统的一部分。服务系统不仅能协调众多用户任务对硬件资源的使用，为用户任务提供一些底层接口（如：创建进程），还能为用户任务提供各种不同的数据处理等高级服务（如：加/解密服务，数据库管理服务等等）。我借鉴了微内核（Microkernel）架构的操作系统[4]，将为用户提供各种服务的进程放入不同的硬服务器中。内核通过服务帧和信息帧与其他内核和硬服务器交互。

2.1 运算系统概述

运算系统为整个系统提供简单运算和复杂数据处理。它与当前通用处理器（如：X86 处理器）的区别是：BZCP 处理器的内核只负责执行一些常用的简单运算指令，复杂运算指令由共享的复杂运算模块执行。

复杂运算指令的执行过程是：线程执行复杂运算指令时，被内核封装成运算帧，并由运算路由器传给相应的复杂运算模块，复杂运算模块接收并完成运算，再将结果封装成信息帧发回给内核的线程。

2.2 服务系统概述

服务系统采用了“机制与策略分离” [4]的思想，实现了原本由微内核提供的多种基础系统服务和一些数据处理等服务，为整个系统提供硬件资源管理和高级服务。在当前主流处理器中，并没有这种概念的硬件模块。服务系统由硬件和软件两部分组成：硬件包含硬服务器模块；软件则表现为运行在内核和硬服务器里的服务线程。

服务系统的服务线程除了提供创建进程，创建线程，进程间通信，内存管理，任务调度，总线驱动等基础系统服务，还可以提供加/解密，数据库管理，图像信号处理（Image Signal Processor，ISP）等数据处理服务。通常提供基础服务的服务线程都运行在硬服务器里，从而解决了阻碍微内核推广的最大问题：性能。

服务系统中，安全机制是很重要的。当前微内核普遍使用权能机制（Capability）[4]加强微内核的安全性。

用户程序通过调用服务指令，以远程过程调用（Remote Procedure Call，RPC）方式来使用服务系统提供

的各种服务。

2.3 一些抽象概念的定义

由于服务系统实现了微内核的功能，在对 BZCP 处理器的详细描述中，需要使用一些抽象概念，如：进程，线程，虚拟地址空间等。我参考了操作系统书籍[5]，对一些抽象概念做简单定义。

进程：进程是系统资源分配的最小单位，每个进程拥有独立的地址空间。进程只是资源的集合，没有“运行态”，“就绪态”等运行状态。进程只有虚拟地址空间，没有实地址空间。每个用户程序至少包含一个进程，每个进程包含一个或多个线程。进程的一些信息，如：进程控制块 PCB（Process Control Block）等，不存放在进程的虚拟地址空间内，而是分散存放在各个硬服务器里。进程使用的所有信息只能存储于同一 BZCP 处理器（可能会存在由多个芯片组合成一个完整的 BZCP 处理器，这些芯片都属于同一 BZCP 处理器）内。进程 ID 的值为 0 有特殊含义：表示任意进程，不能分配给进程。

线程：进程是线程的载体容器。进程内的多个线程共享进程的地址空间和资源。内核执行和调度的对象是线程。线程只有“用户态”，没有“内核态”。线程的一些信息，如：线程控制块 TCB（Thread Control Block）等，不存放在进程的虚拟地址空间内，而是分散存放在各个硬服务器里。进程内各个线程间可以通过信号量,消息队列等方式实现线程间的同步和互斥操作。线程 ID 的值为 0 有特殊含义：表示任意线程，不能分配给线程。

服务进程：包含了服务器线程或者被观察者线程（详情在 7.2 中介绍）的进程。必要或者频繁使用的服务进程一般运行在硬服务器里；其它服务进程运行在内核里。

服务线程：所有的服务功能都由服务线程提供。有两种服务线程：服务器线程和被观察者线程。

虚拟地址空间：虚拟地址空间是存放进程信息的非负整数地址的有序集合。每个进程的虚拟地址空间只有“用户空间”（“user space”），没有“内核空间”（“kernel space”）。虚拟地址空间的大小在进程创建时定义，但在 N 位处理器中，进程的虚拟地址空间不能超过 2^N 字节。如：在 64 位处理器中创建进程时，可以将虚拟地址空间设置为 16 位，此进程空间大小为 2^16 字节，地址为 0 - （2^16 – 1）；也可以将虚拟空间设置为最大值 64 位，此进程空间大小为 2^64 字节，地址为 0 - （2^64 – 1）。

帧包：内核与其他模块都是通过帧包进行交互的。帧包由帧头，数据段，帧尾三个部分组成。帧头包含路由等信息，数据段是帧包传输的内容，帧尾包含校验码等信息。封装了运算指令的帧包称为运算帧；封装了服务指令的帧包称为服务帧；其他包含应答，返回，错误等信息的帧包统称为信息帧。帧包的字节长度比较短，数据段和部分路由信息都能存放在线程使用的寄存器里。（包含大量数据的信息只能在硬服务器之间传输，对用户程序透明，详见 7.4 节）

组进程：组进程是一种特殊的用户进程。它作为进程的容器，在逻辑结构上可以包含多个进程。组进程内的进程之间能进程间通信（Inter-process communication，IPC），不在一个组进程的进程不能直接进行进程间通信（IPC）。组进程是可以嵌套的，组进程 A 里创建了组进程 B，则组进程 A 称为组进程 B 的父组进程，组进程 B 称为组进程 A 的子组进程。子组进程以进程的逻辑表象存在于父组进程里，子组进程内的进程经过子组进程端口转换后可以与父组进程里的其他进程进行进程间通信（IPC）。每个组进程内都有自己的运行时环境（Runtime Environment）。

BZCP 处理器只是 ZSSC 架构下的一种简单实现模型，因此处理器中的所有用户进程都属于同一个组进程。

2.4 BZCP 处理器整体结构的总结

BZCP 处理器里，内核区域通过不同的运算帧和服务帧与运算区域和服务器区域实现数据交互。即内核中只处理一些简单操作，其他操作都被封装成帧包发送给相应的模块处理。由此，实现了 ZSSC 架构中运算和服务的共享设定。关于此操作的具体实现方式，将在后面详细说明。

BZCP 处理器能适应从嵌入式控制器到超级计算机等各种规模、不同等级的运算需求。但是 BZCP 处理器的结构比较复杂，并不适应最袖珍的嵌入式控制器，一般需要 Linux 操作系统的嵌入式应用可以尝试使用 BZCP 处理器。

3 BZCP 处理器的存储系统结构

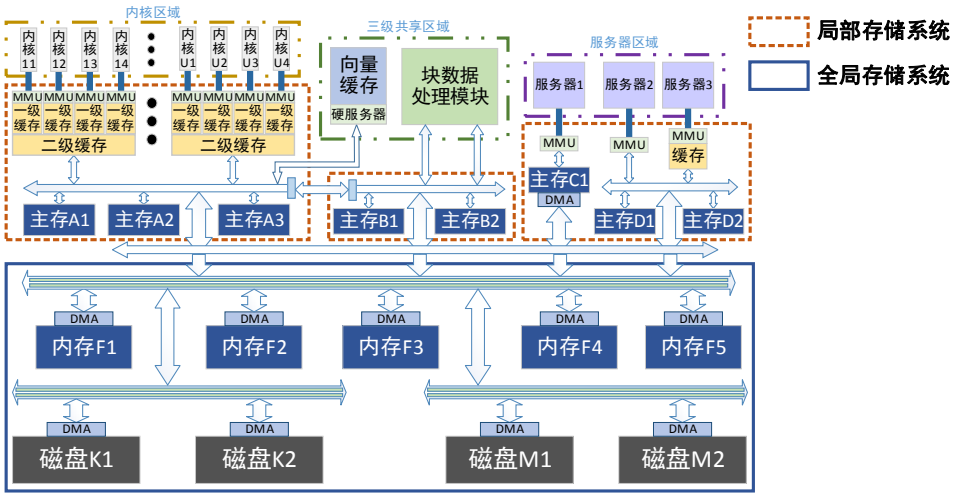


图 3.1 BZCP 处理器的存储系统结构（忽略了各存储区域内的硬服务器）

如图 3.1 所示，BZCP 处理器的存储系统由三个局部存储模块和一个全局存储模块组成。

BZCP 处理器中有三个区域需要与存储系统连通，它们分别是“内核区域”，“三级共享区域”，“服务器区域”。其中，“三级共享区域”中的向量缓存和内核区域连接同一个局部存储模块；“三级共享区域”中的“块数据处理模块”连接一个局部存储模块；所有服务器模块连接一个局部存储模块。

三个局部存储模块都有各自独立的地址空间。每个局部存储系统都包含有至少一个硬服务器。通过硬服务器，三个局部存储模块可以间可以相互直接通信，并能自主访问全局存储模块。

与内核区域连接的局部存储系统中，MMU 模块和一，二级缓存可以与当前通用处理器（如：X86 处理器）一样，放置在处理器芯片内。主存（A1，A2，A3，B1，B2，C1，D1，D2）可以通过高带宽存储器（High Bandwidth Memory，HBM）技术与芯片封装在一起。“内存 Fn”（n=1，2，3，4，5）可以是当前常用的 SDRAM，如：DDR4 SDRAM (Double Data Rate 4 SDRAM)。磁盘（K1，K2，M1，M2）是当前常用的磁盘存储器或者固态硬盘（Solid State Disk 或 Solid State Drive，SSD）等存储设备。

3.1 与内核区域相连的局部存储系统

图 3.1 中与“内核区域”和“三级共享区域”中向量缓存相连的局部存储系统包含了“MMU”，“一级缓存”，“二级缓存”，“主存 An”（n=1，2，3）模块。“主存 An”（n=1，2，3）组成的存储区域有自己独立的地址空间。

此局部存储系统主要为运行在内核中的线程提供缓存。它与当前通用处理器（如：X86 处理器）的处理方式相似，可以参考计算机书籍[6]。但是，每个缓存模块中都存在多个线程的缓存，每个 MMU 模块都需要能同时将多个线程的虚拟地址转换成物理地址，这对缓存和 MMU 的结构设计提出新的挑战。

向量数据的连续多字节操作不适应传统各级缓存的存储器层次结构（memory hierarchy），所以在 BZCP 处理器中，向量处理模块经过向量缓存，直接对局部存储空间进行读写操作。

内核区域的局部存储系统中，存在多个硬服务器。这些硬服务器可以和内核直接通信。它提供如下服务：

1. 管理进程虚拟地址使用的存储空间。
2. 当出现缺页（page fault）等情况时，由硬服务器处理，处理完后，正常完成指令操作。
3. 线程读写存储器操作异常（如：对只读存储空间执行写操作），则发送异常信息给线程所在的内核。
4. 完成向量缓存和主存之间传输向量数据的控制。
5. 与其他两个局部存储系统的硬服务器协同工作，实现大量数据的传输。
6. 局部存储系统内部，大量数据迁移、复制等操作。
7. 与全局存储模块的数据交互。

3.2 与“块数据处理模块”相连的局部存储系统

图 3.1 中与“块数据处理模块”相连的局部存储系统包含了 “主存 Bn”（n=1，2）模块。“主存 Bn”（n=1，2）组成的存储区域有自己独立的地址空间。

此局部存储系统只有“块寄存器”（6.3 节中介绍）一种类型的数据（其他数据类型的数据如：标量数据、遮罩数据等，也是以“块寄存器”的数据类型进行存储，多个其他数据类型的数据可以存放在同一个“块寄存器”

里）。“块数据处理模块”读写局部存储器的地址都是虚拟地址，其映射到局部存储的物理地址由此局部存储系统的硬服务器管理。

此局部存储系统中存在一个硬服务器。它可以和“块数据处理模块”直接通信。它提供如下服务：

1. 管理“块执行段”（4.5 节中介绍）中“块寄存器”使用的存储空间。
2. 与其他两个局部存储系统的硬服务器协同工作，实现大量数据的传输。
3. 局部存储系统内部，大量数据迁移、复制等操作。
4. 与全局存储模块的数据交互。

3.3 与服务器区域相连的局部存储系统

图 3.1 中与“服务器区域”相连的局部存储系统包含了“MMU”，“主存 C1”，“主存 Dn”（n=1，2）模块。“主存 C1”有自己独立的地址空间。“主存 Dn”（n=1，2）组成的存储区域也有自己独立的地址空间。

服务器区域的局部存储系统中，存在至少一个硬服务器。这些硬服务器可以和服务器区域里对应的硬服务器直接通信，为硬服务器提供存储空间。“主存 C1”和“主存 Dn”（n=1，2）有各自的服务线程，这两个服务线程可以分别在两个硬服务器里运行，也可以在同一个硬服务器中运行。

服务器区域的局部存储系统与内核区域的局部存储系统一样，都是为各种线程提供服务的，所以他们的硬服务器提供的服务也是类似的。

3.4 全局存储系统

图 3.1 中与三个局部存储系统连接的 “内存 Fn”（n=1，2，..，5），“磁盘 Kn” （n=1，2），“磁盘 Mn” （n=1，2）模块构成了全局存储系统。“内存 Fn”（n=1，2，..，5）组成的存储区域有统一并独立的地址空间。

全局存储系统中，至少存在一个硬服务器。此硬服务器可以和三个局部存储系统直接通信。它主要提供数据暂存服务和永久存储服务。当各局部存储系统中的存储空间不够时，将一些可能短时间不会使用数据暂存在全局存储系统中，等需要时再取出。一些断电后依然需要长期存储的数据也会被存入全局存储系统中。

4 指令集

在 BZCP 处理器里，指令集是运行在内核里的线程用来计算、通信和控制计算机系统的一套指令的集合。它是一种指令长度不定，包含各种运算和服务的指令集，是一种复杂指令集。我将这种指令集架构命名为祝氏指令集架构(Zhu’s Instruction Set Architecture，ZISA)。

在 BZCP 处理器里，硬服务器模块（详见 7.2 节）和“块数据处理模块”（详见 6.3 节）有自己的指令集和处理器架构，其指令集与 ZISA 指令集没有关系。

ZISA 指令集参考 RISC_V[7]的方式将指令集架构（ISA）模块化。BZCP 处理器里的指令集分成了基础指令集，标量扩展指令集，向量指令集，块指令集，服务指令集五个部分。

为了适应从嵌入式控制器到高性能计算机等各种规模的运算需求，应将各部分的指令集分解为多个模块，如：将标量扩展指令集分解为四精度浮点指令集，三角函数指令集，十进制浮点指令集等多种指令模块。在设计处理器时，可以根据使用环境实现相应模块的指令集。

不同位数处理器的寄存器宽度和相应的用户地址空间大小都不相同。基础指令集有三种主要的变种对应不同位数处理器：对应 32 位处理器的 ZA32b 指令集；对应 64 位处理器的 ZA64b 指令集；对应 128 位处理器的 ZA128b 指令集。ZISA 指令集有向下兼容的特点，所有 32 位处理器上能执行的指令，可以直接在 64 位处理器或者 128 位处理器上执行。

我只能对 ZISA 指令集进行简略的描述，并不能构造一个完整的 ZISA 指令集。本章列举了一些后面章节会使用的指令助记符（不包含二进制的指令码）。

4.1 寄存器

BZCP 处理器模型有 32 个通用寄存器，程序计数器 PC 和标志寄存器 status。整数和浮点数共用通用寄存器。

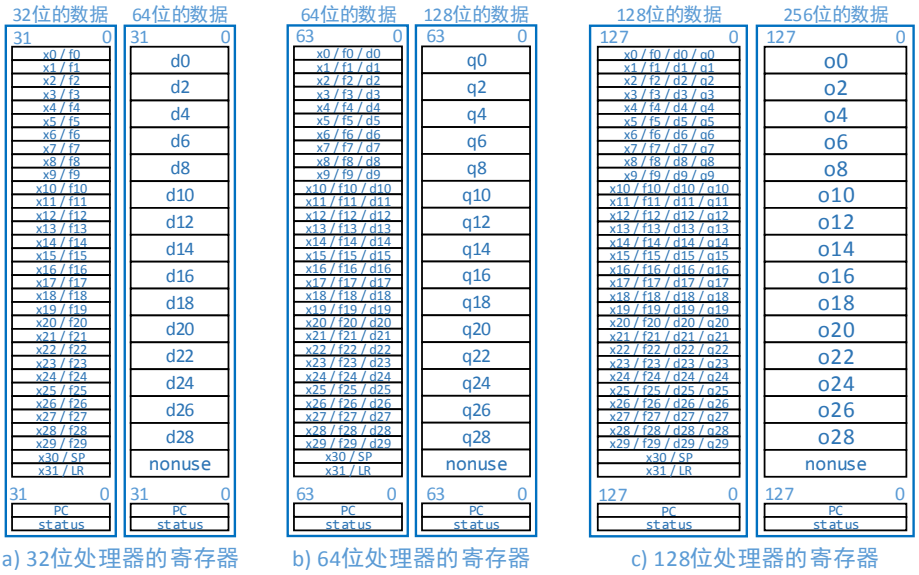


图 4.1 不同位数处理器的寄存器

图 4.1 中，xn(n=0…31)是通用寄存器，其位数与处理器位数相同。fn(n=0…29) 存放 32 位浮点数；dn(n=0…29) 存放 64 位浮点数；qn(n=0…29) 存放 128 位浮点数；on(n=0,2,4,…,28) 存放 256 位浮点数。所有存放在寄存器中的数据都采用小尾端模式的存储方式。

整数和浮点数的寄存器分布方式是为了实现不同位数处理器的向下兼容。如：在 32 位处理器上，程序使用了浮点数 d14，就不能同时使用通用寄存器 x14 和 x15，这与 64 位处理器上寄存器的使用情况是兼容的。因此，当这段程序在 64 位处理器上运行时，也能得到一致的结果。

为了实现不同位数处理器的向下兼容，不能通过对通用寄存器 Xn 的操作来处理相应位置的浮点寄存器。

4.2 基础指令集 V0.1

基础指令是完成一些简单操作的指令，一般都在内核中执行。比如：32 位处理器的基础指令集完成的功能与 RISC_V 中的 RV32IMFDC[7]基本相同；64 位处理器的基础指令集完成的功能与 RISC_V 中的 RV64IMFDC[7]基本相同。

在基础指令集中，一般尽可能使用 16 位指令编码来实现其中使用频率高的指令。

表 4.1 部分基础指令

指令	示例	含义	注释
ADD[U].I32	add.i32 x5, x6, x7	x5 = x6 + x7	属于 ZA32b。在 64 位或 128 位处理器中运行时，会将结果中的 32 位写入进位标志位，并对 32 位到最高位进行[符号位 / 0]扩展。
ADD[U].I64	add.i64 x5, x6, x7	x5 = x6 + x7	属于 ZA64b。在 128 位处理器中运行时，会将结果中的 64 位写入进位标志位，并对 64 位到 127 位进行[符号位 / 0]扩展。
ADD[U].I128	add.i128 x5, x6, x7	x5 = x6 + x7	属于 ZA128b。
UEF{I8/I16/I32}	uefi8 x3, x4	x3 = 无符号扩展(x4[7:0])	属于 ZA32b。对数据中的低{8/16/32}位进行无符号扩展
UEFI64	uefi64 x3, x4	x3 = 无符号扩展(x4[63:0])	属于 ZA64b。在 128 位处理器中运行时，对数据中的低 64 位进行无符号扩展。对于 64 位处理器，视为 nop 指令。
SEF{I8/I16/I32}	uefi8 x3, x4	x3 = 有符号扩展(x4[7:0])	属于 ZA32b。对数据中的低{8/16/32}位进行有符号扩展
SEFI64	uefi64 x3, x4	x3 = 有符号扩展(x4[63:0])	属于 ZA64b。在 128 位处理器中运行时，对数据中的低 64 位进行有符号扩展。对于 64 位处理器，视为 nop 指令。
BLEZ	blez x5, go_break	x5 小于等于 0 则跳转到 go_break	属于 ZA32b。条件跳转语句。
BNEZ	bnez x5, go_break	x5 不等于 0 则跳转到 go_break	属于 ZA32b。条件跳转语句。
J	j go_break	无条件跳转到 go_break	属于 ZA32b。无条件跳转语句。
RET	ret	从子过程返回	属于 ZA32b。

4.3 标量扩展指令集 V0.1

所有不属于基础指令集的标量指令都归属于标量扩展指令集。标量扩展指令通常在共享区域中执行，但是，为了提高在特定应用场景的运行速度，会在设计处理器芯片时，将频繁使用的复杂标量运算模块也放在内核中。

线程执行标量扩展指令时，被内核封装成运算帧，发送到相应的标量模块中执行。发送运算帧后，线程进入

“阻塞状态”。标量模块执行完成后，将结果传回原线程。其执行过程详见 6.1 节。

借鉴 RISC-V，标量扩展指令集也分成多个扩展指令模块。这些看着指令模块可以参考 RISC-V 中的四精度浮点指令集，十进制浮点指令集，三角函数指令集等。

表 4.2 部分标量扩展指令

指令	示例	含义	注释
FSIN.{F32/F64/F128}	fsin.f64 d2, d4	d2 = sin(d4)	返回 64 位浮点数 d4 的正弦值，并赋值给 d2。

4.4 向量指令集 V0.1

向量指令是在向量处理模块中执行的指令。每个线程在共享的向量处理模块中拥有一个向量寄存器组，向量寄存器组中包含 32 个向量寄存器和 1 个向量状态寄存器 mstatus。向量寄存器表示为 vn(n=0…31)。向量指令集参考了 RISC-V 中的向量指令集[7]，可先阅读 RISC-V 的向量扩展规范，再看 BZCP 处理器的向量操作。

向量指令被封装成运算帧，再发送到向量处理模块中执行，发送运算帧后，线程进入“阻塞状态”。向量处理模块执行完成后，会将执行结束标志传回原线程。向量模块详细的介绍和运行过程在 6.2 节介绍。

表 4.3 部分向量指令

指令	示例	含义	注释
VADD.VV	vadd.vv v9, v5, v1	v9 = v5 + v1	必须先执行设置元素位数、个数等参数的指令，然后才能正确执行此指令。
VDEL	vdel	释放当前线程使用的向量寄存器组	当一段向量运算函数运算完后，使用此指令，可以直接释放此线程占用的向量寄存器组。下次需要使用时，再重新分配具有默认初始值的向量寄存器组。

4.5 块指令集

块指令是发送到“块数据处理模块”执行的指令。块指令被封装成运算帧，再发送到“块数据处理模块”。

与其他运算模块不同，“块数据处理模块”是完成大量数据高速运算的模块，它内部运行的指令集和 ZISA 指令集是两种不同的指令集。“块数据处理模块”接收完一段完整的“块执行段”，然后将其翻译为本模块内部使用的指令段，并分配一个线程来执行此指令段。

```
块标号 : 处理器位数 : block  ; 此行对应块指令为：b.start  #immed_8, #immed_32
                             ; immed_8为处理器位数, 值=处理器位数/8(32位处理器的值为0x04)
                             ; immed_32为块标号
    块指令
    基础指令
    块指令
end block [块标号] ;此行对应块指令为：  b.end   #immed_32  ; immed_32为块标号
```

图 4.2 完整的块执行段形式

完整的“块执行段”如图 4.2 所示，它由开始指令 b.start（包含开始指令）到结束指令 b.end（包含结束指令）的所有指令组成。块指令只能出现在“块执行段”中。内核中的线程一条一条运行指令，遇到块指令就将其封装并发送到“块数据处理模块”。执行到 b.end 指令时，指令被封装成运算帧并发送出去后，线程进入“阻塞状态”。当内核收到“块数据处理模块”执行完毕的返回消息帧时，才会重新将对应线程切换到“就绪状态”。

“块执行段”中可以包含不是块指令的指令，这些“非块指令”可以在内核正常执行，不会被发送到“块数据处理模块”。但“非块指令”有两点限制：1.不能使块指令循环（块指令集中包含了实现循环的指令）；2.不能使用块指令的运算结果。还需要注意“非块指令”是否会产生异常，异常是否会影响“块执行段”的正常运行。建议在“块执行段”中只包含块指令和简单的基础指令。

块指令与“块数据处理模块”的结构息息相关，它的具体信息放在“块指令集 V0.1”（6.3.3 节）中描述。

4.6 服务指令集

服务指令是程序与服务系统之间的接口。线程执行服务指令时，内核会将指令封装成服务帧，再由服务路由器发送到相应的服务线程中处理。服务线程接收并完成相应的操作，会将结果封装成信息帧发回给线程。内核里的线程和硬服务器里的线程既可以使用服务指令，也可以申请成为服务线程。

服务指令的形式和执行过程与服务系统息息相关，它的具体信息放在“服务指令集 V0.1”（7.3 节）中描述。

5 内核模块

内核区域包含处理器内所有内核模块。内核模块的主要功能是运行线程指令和线程调度。所有的用户任务都在内核模块中运行。BZCP 处理器包含的内核模块数量可以很多，多达成百上千个。

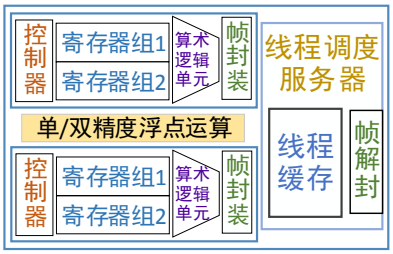


图 5.1 内核模块的结构图

如图 5.1 所示，内核模块中包含了线程子服务器，精简处理器模块，单/双精度浮点运算模块。其中，单/双精度浮点运算模块由两个精简处理器模块共享使用。下面，将详细介绍其中一些模块。

5.1 精简处理器模块

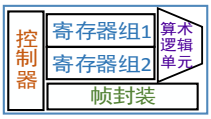


图 5.2 精简处理器模块的结构图

精简处理器模块是一个结构简单的微处理器，由算术逻辑单元（Arithmetic&logical Unit，ALU），寄存器组，控制器，帧封装等模块组成，它的功能是执行一些简单的指令，并将在外部运行的指令（复杂运算指令和服务指令，都是不在精简处理器模块中执行的指令）封装成帧包，发送出去。

5.1.1 精简处理器模块的基本信息

精简处理器模块通过流水线方式执行取指令（Instruction fetch，IF），指令解码（Instruction decode，ID），执行（Execute，EX），存储器访问（Memory access，MEM），回写（Write back，WB）操作。在指令解码后，当指令为在外部运行的指令（复杂运算指令和服务指令）时，控制器会将相应的数据传递给“帧封装”模块，“帧封装”模块将一些参数和地址等数据封装成运算帧或者服务帧，然后发送到相应的路由器。“帧封装”模块封装数据的操作称为“指令封装”（Instruction package，IP）。“指令封装”操作时获取寄存器数据的个数不固定，执行的时钟周期也不固定。

BZCP 处理器中内核的个数很多，能并行处理多个线程，除了能在单位时间内并行执行很多条指令，还能为其他模块提供足够多的外部运行指令（复杂运算指令和服务指令），提高其他模块的使用率。精简处理器模块可以采用当前一些成熟的技术来提升运行速度，如：超标量（Superscalar）技术，超流水线（Superpipeline）技术，乱序执行（Out-of-order Execution）技术，高级分支预测（Advanced Branch Prediction）技术等。这些技术是否采用需要综合考量芯片的应用场景，芯片制造技术，封装技术等因素。

BZCP 处理器只是 ZSSC 架构下的一种简单实现，因此，精简处理器模块非常简单。在指令解码后，只有当指令中的数据不存在写后读（RAW）相关，读后写（WAR）相关，写后写（WAW）相关，才会执行此指令。

精简处理器模块中包含两个寄存器组，能同时保存两个线程的状态信息。两个线程以细粒度多线程（Fine-Grained Multithreading）（参见 Sun T1 处理器）方式交替执行。

BZCP 处理器中内核的个数很多，而且每个内核可以同时运行 4 个线程，造成每个线程能分配到的一级和二级缓存都相对较少，线程存取数据所耗的平均时间会比当前通用处理器（如：X86 处理器）大很多。加上精简处理器模块简单的结构，造成精简处理器模块可能在很多时候处于空闲状态。为了提高精简处理器模块的使用效率，建议采取如下几种方式：

1. 增大线程的寄存器个数，比如采用 64 个通用寄存器，减少对存储器的读写次数。
2. 精简处理器模块可以包含更多的寄存器组。

3. 代码上，减少连续指令间的寄存器依赖。
4. 缩短线程切换的时间。可以将切换前后，两个线程的寄存器组都缓存在线程子服务器中的线程缓存中。
5. 能读写连续多个寄存器数据。比如一个指令将多个寄存器数据读出/写入存储器。

5.1.2 精简处理器模块的异常和中断

精简处理器模块将不能执行的指令发送出去处理，因此，异常除了发生在模块内，还会发生在其他模块上，并且内核的个数很多，不适合使用通用中断控制器（Generic Interrupt Controller，GIC）来处理中断。

当线程在精简处理器模块内部执行指令时遇到异常的异常称为内部异常。发生内部异常后，由线程子服务器控制精简处理器模块，保存异常线程的运行环境，再根据异常向量表跳转到相应的异常处理程序处执行。

当线程执行在外部运行的指令（复杂运算指令和服务指令）时，执行指令时发生的异常称为外部异常。如：取数据异常，指令运算异常等。发生外部异常时，外部模块会将异常信息封装成服务帧发送给相应的内核。内核的线程子服务器收到服务帧后，判断发生异常的线程是否占用精简处理器模块的寄存器组，若线程正在占用，会先保存线程的运行环境；若线程没有占用，则需要先将一个正在运行的线程调度出去，腾出一个寄存器组资源。再将服务帧中的信息写到相应的寄存器中，最后根据异常向量表跳转到异常处理程序处执行。

线程的中断都是通过异步进程间通信（异步进程间通信详细过程见 7.6 节）的方式产生。当线程调用异步通信指令关注其他进程中的事件主题时，会设置一个中断编号值：vid，此 vid 值需要对应本线程的一个中断处理程序。当事件满足触发条件时，被观察者线程会发送一条包含 vid 值的服务帧给本线程所在的内核，内核的线程子服务器收到服务帧后可以根据 vid 值和其他信息，直接计算出中断处理程序的起始地址，进而使线程进入中断状态，并将服务帧中的信息写到相应的寄存器中，然后跳转到中断处理程序。

中断和异常的处理方式差不多，区别在于异常是根据异常向量表的值跳转，而中断则是根据服务帧中包含的 vid 值跳转到相应的中断处理程序。由于有线程子服务器的参与，线程在执行异常或中断处理程序时，都可以从通用寄存器中获得一些与异常或中断有关的信息。这与当前通用处理器的中断处理方式是不同的。

当线程执行在外部运行的指令（复杂运算指令和服务指令）时，线程进入“阻塞状态”。如果此时接收到会进入中断的服务帧，线程会进入中断处理程序。在中断处理程序返回前，使线程阻塞的指令处理完并返回了信息帧。线程子服务器接收并处理此信息帧，还原线程的运行空间并将返回信息赋值给线程，然后再将此线程移出内核，并将线程状态由“阻塞状态”修改为“就绪状态”，等待中断返回后，才能被再次调度。

5.2 线程子服务器

线程子服务器主要完成三种功能：1.控制调度内核中的线程；2.接收并执行来自线程服务器的服务帧，完成线程的创建，删除和转移等操作；3.接收，解析并处理发送给线程的服务帧。

线程服务器通过服务帧给线程子服务器一些线程调度的控制参数，线程子服务器根据控制参数设置调度的方式（优先级或者时间片或者混合），时间片的长短等。每个线程子服务器可以设置不同的控制参数。

如果处理器内各个内核的负载不平衡，线程服务器就会进行整体调整。通过发送不同的服务帧给两个相关的线程子服务器，能将一条线程从负担重的内核转移到负担轻的内核。一些相关资源也需要迁移，比如：线程的向量寄存器组等。

线程执行在外部运行的指令（复杂运算指令和服务指令）后，一般会被线程子服务器转移出精简处理器模块。线程被经常调度，并且每次使用的寄存器组都是随机的。因此，所有发送给线程的帧包，都不能由寄存器组作为目的地址，只能将线程子服务器作为目的地址。所有发送给线程的帧包会被发送到相应的线程子服务器，线程子服务器接收并解析帧包后，会将信息写入线程当前使用的通用寄存器和状态寄存器里。如果当前线程不在“运行态”，则会先将线程调度到精简处理器模块后，再做相应操作。

线程子服务器内部有缓存，当缓存被占满后，接收到的服务帧或者信息帧都会送到局部存储器中缓存。

6 运算系统

运算系统是由内核和各种运算模块组成。运算模块分成三种类型：复杂标量运算模块，向量处理模块，块数据处理模块。下面将分别介绍这些类型的模块。

6.1 复杂标量运算模块

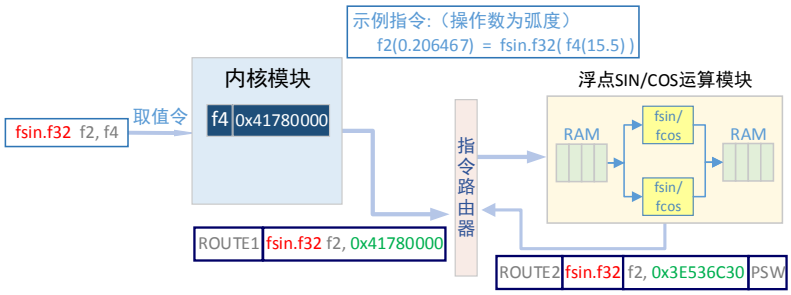


图 6.1 复杂标量模块的运算流程

复杂标量运算模块执行 SISD 类型指令的模块。它的特点：运算模块接收到的运算帧里包含运算方式和运算数据。

图 6.1 所示为单精度浮点 SIN/COS 运算的流程。线程取得指令“`fsin.f32  f2,f4`”，发现是复杂运算指令，则将指令操作数从寄存器中取出，由内核封装成运算帧发给“浮点 SIN/COS 运算模块”。“浮点 SIN/COS 运算模块”接收并计算出结果后，将结果封装成信息帧发送给相应的内核。内核里的线程子服务器接收到信息帧并取出结果后，更新线程的通用寄存器和状态寄存器（依据 PSW 字段中的标志位）。

复杂标量运算模块的计算过程比内核内部的运算过程要复杂很多，尤其在发送和接收时，都有封装，解封的操作。其指令运算速度会比内核内部执行的指令慢很多。在特定应用场景中，有一些复杂运算指令会被频繁调用。为此，可以将这些复杂运算模块包含进内核中，以提高程序的运行速度。

6.2 向量处理模块

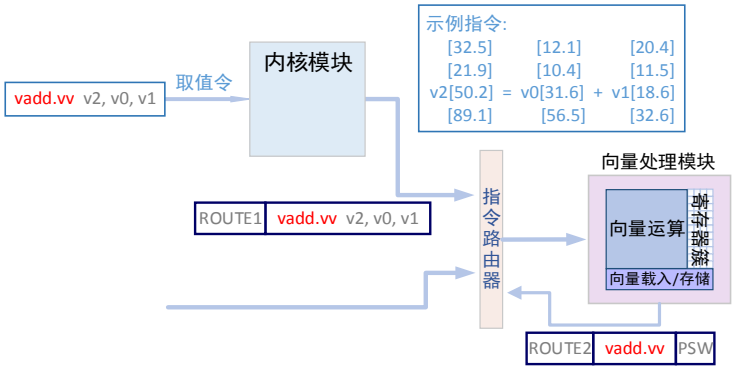


图 6.2 向量模块的运算流程（假设已经设置了向量数据类型和宽度）

向量处理模块是执行单指令流多数据流（Single Instruction Multiple Data，SIMD)类型指令的模块。它的特点：线程的向量寄存器组存放在向量处理模块里。

图 6.2 所示为向量浮点加的运算流程。向量处理模块与复杂标量运算模块的运行流程一样：线程取得向量浮点加指令，由内核将指令封装成运算帧再发给向量处理模块。向量处理模块计算完成后，将结果封装成信息帧发送会相应的内核，内核里的线程子服务器接收到信息帧并取出结果后，更新相应线程的状态寄存器（依据 PSW 字段中的标志位）。

向量处理模块中的“寄存器簇”（Reg Cluster）可以存放多个向量寄存器组。每个使用了向量运算的线程都拥有一个向量寄存器组，这些寄存器组就存放在向量模块的“寄存器簇”中。这种将所有线程的向量寄存器组放在向量处理模块的方案相比于放在内核里的方案有三个优点：1.降低了内核结构的复杂度。2.减少线程切换时保存向量寄存器的操作。3.不需要将向量数据封装进运算帧。

向量寄存器组中的数据可以通过指令与内核交互，也可以由向量模块直接存取到局部存储系统。前者适合对少量数据的读写，后者适合对整个向量寄存器组的读写操作。由于很多线程共用同一个向量处理模块，会出现本线程在执行向量指令时，“寄存器簇”被其他线程占满，没有可用向量寄存器组的情况。遇到这种情况，向量处理模块会将“寄存器簇”中一个不常使用的向量寄存器组移到“三级共享区域”的向量缓存中，再将清空的向量寄存器组分配给本线程使用。当向量缓存的存储空间也被占满时，可以将数据转移到局部存储系统。

关于向量寄存器组的结构，向量处理模块的操作，如：向量数据条件执行等，可以参考 RISC-V 中关于向量扩展的部分[7]，并根据 BZCP 处理器的特点进行优化。

6.3 块数据处理模块

“块数据处理模块”是完成大量数据高速运算的模块，在 BZCP 处理器里承载着集中提供强大算力的职责。

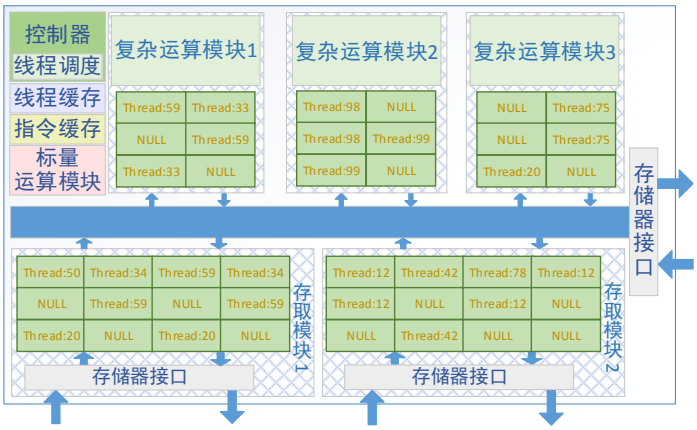


图 6.3 “块数据处理模块”结构

如图 6.3 所示：“块数据处理模块”由控制器，多个复杂运算模块，各种数据缓存模块等组成。它有自己独特的指令集架构（ISA）和运行方式，可以看成一个独立的数据流计算机。

6.3.1 块寄存器

“块数据处理模块”内部采用寄存器方式存取数据和记录运算的临时数据。“块数据处理模块”中存储大量数据的寄存器是一块连续存储空间，称为“块寄存器”。“块寄存器”是对大量数据读写、转移和运算的基础存储单元。“块寄存器”内包含多个相同数据类型的数据，数据的数据类型由指令设置为整数或者浮点数。“块寄存器”的空间大小会根据生产工艺，应用场景等因素而决定。但在同一“块数据处理模块”内，每个“块寄存器”的存储空间是相同的。

“块数据处理模块”以“块寄存器”为数据单元，将其传输到“复杂运算模块”里，经过“复杂运算模块”内的数据处理，再把结果输出到“块寄存器”。以此来实现了单指令流多数据流（SIMD)的运算模式。

6.3.2 复杂运算模块

“块数据处理模块”内部包含至少一个“复杂运算模块”，这些“复杂运算模块”是高速处理大量数据的核心。每个复杂运算模块都可以拥有各自不同的运算模型，只需运算模型包含数据输入/输出接口（一般情况下还需包含控制接口），就能封装成一个复杂运算模块。

这些“复杂运算模块”可以是通用的矩阵运算模块，可以是大规模数据仓库中检索特定数据和数据挖掘，也可以是特定算法的硬件实现模块（如：加/解密算法，视频编/解码算法，存算一体的模块），甚至可以是与其他运算器（如：量子计算机）通信的接口模块。在实现方式上，特殊的“复杂运算模块”可以使用 FPGA（Field Programmable Gate Array）实现。

这种处理模式可以将图像处理器（Graphics Processing Unit，GPU），张量处理器（Tensor Processing Unit，TPU），神经网络处理器（Neural network Processing Unit，NPU）等大数据并行运算的硬件模块转变为“复杂运算模块”，进而融入到通用处理器结构里。即可以针对特定应用领域创建特定的“复杂运算模块”。

当需要某种“复杂运算模块”频繁执行指令时，可以通过在“块数据处理模块”里增加这种“复杂运算模块”的个数来分担运算需求。

6.3.3 块指令集 V0.1

块指令是在“块数据处理模块”中执行的指令。由块指令组成的“块执行段”被完整发送到“块数据处理模块”后，“块数据处理模块”会将“块执行段”翻译成本模块使用的指令段，再运行。

从“块数据处理模块”的结构就可以看出，使其高速运算的关键之一就是不间断地为各个“复杂运算模块”提供数据，并尽快运行。这种数据驱动方式与数据流计算机[1]很相似。因此，在块指令的设计时，将“块数据处理模块”内部设定为数据流计算机。与基础指令集不同，块指令的输入数据在左边，输出数据在右边。

块指令中有三种类型的数据：第一种是标量数据，包含整数和浮点数。在指令中使用内核寄存器的表示方式：Xn（n=1,2,…），Fn（n=1,2,…）。指令翻译后，标量数据一般被放在指令中。第二种是块数据，在指令中使用

Bn（n=1,2,…）表示，它是“块寄存器”的概念拓展，是一个容量没有限制的“块寄存器”。指令翻译后，块数据被分段，每段数据都被存放在一个“块寄存器”里。第三种是遮罩数据，在指令中使用 Mn（n=1,2,…）表示，遮罩数据的含义参考书[6]。遮罩数据与对应块数据关联，对应块数据中有多少个运算数据，遮罩数据中就有多少个 bit 位。指令翻译后，遮罩数据也被分段，与被分段的块数据一一对应。分段后的遮罩数据会被存放在“块寄存器”里，一个“块寄存器”可以存放多个遮罩数据。

“块数据处理模块”中的块指令分成两部分：一部分是通用指令，是“块数据处理模块”必须实现的指令，包含标量指令，简单的存储器读写指令和控制指令等；另一部分是私有指令，是“块数据处理模块”可以自由增减的块指令，包含复杂的存储器访问指令集、置换指令集和复杂运算指令集等。私有指令可以分成多个模块化的指令集。每种“复杂运算模块”都有属于自己的专属指令集。

为了软硬件兼容，我们需要对通用指令制定一套“通用块指令集标准”，对私有指令制定模块化的“功能模块指令集标准”。每种“功能模块指令集”包含一个编号和多条块指令。“块数据处理模块”中的每个功能模块必须能执行一个（或者多个）“功能模块指令集”中所有的块指令。线程通过查询能够获得本芯片全部的“功能模块指令集”编号，从而获得“块数据处理模块”能够执行的所有私有指令。

表 6.1 部分通用块指令集

标准指令	标准指令示例	含义	注释
B.START	b.start #0x04, #0x14586200	使用 32 位处理器，“块执行段”的开始指令。	运行“块执行段”之前，需要检查硬件系统是否能执行指令段内的所有指令。(指令参数含义见图 4.2)
B.END	b.end #0x14586200	“块执行段”的结束指令。	线程发送此指令成功后会进入“阻塞状态”。
B.SET2	b.set2 u32, u32, X0, X1, oX0, oX1	oX0 = X0; oX1 = X1; (x0,x1 都是 32 位无符号整数)	通过此指令，内核可以向“块数据处理模块”传输标量数据。
B.LD.type	b.ld.f32 [iX0], iX1, oB0	从地址 iX0 开始读取 iX1 个数据类型为 f32 的数据，存放在块数据 oB0 中。	块数据 oB0 的存储空间没有限制。iX0，iX1 是与处理器位数相同的无符号整数。iX1 不能为零。
B.ST.type	b.st.f32 iB0, iX0, [iX1]	将块数据 iB0，以 iX0 个数据类型为 f32 的形式存放到地址 iX1 开始的存储器中。	iX0，iX1 的数据类型为处理器的位数。iX0 不能为零。
B.NOP1.TYPE	b.nop1.f32 iX0, oX0	oX0 = iX0。	块指令在使用过程中，产生数据的指令和使用数据的指令相隔很远，超过了相对位置编码的范围，则需要此指令实现数据中转。（iX0 和 oX0 的数据类型为 f32）
B.MARKNOP1	b.marknop1 iM0, oM0	oM0 = iM0。	
B.BLOCKNOP1	b.blocknop1 iB0, oB0	oB0 = iB0。	
B.MARKNOP2	b.marknop2 iM0, iM1, oM0, oM1	oM0 = iM0; oM1 = iM1。	
B.MARKNOP4	b.marknop4 iM0, iM1, iM2, iM3, oM0, oM1, oM2, oM3	oM0 = iM0; oM1 = iM1; oM2 = iM2; oM3 = iM3。	
B.COPY2.TYPE	b.copy2.u32 iX0, oX0, oX1	oX0 = iX0; oX1 = iX0。	将 u32 类型数据 iX0 复制给 2 个输出节点。
B.MARKCOPY2	b.markcopy2 iM0, oM0, oM1	oM0 = iM0; oM1 = iM0。	将 iM0 复制给 2 个输出节点。
B.RT	b.rt iB0, iX0, oB0	if(iX0) { oB0 = iB0; }	T 门控节点[1]
B.RF	b.rf iB0, iX0, oB0	if(! iX0) { oB0 = iB0; }	F 门控节点[1]
B.SWITCH	b.switch iB0, iX0, oB0, oB1	if(iX0) { oB0 = iB0; } else { oB1 = iB0; }	开关门控节点[1]
B.MERGE	b.merge iB0, iB1, iX0, oB0	if(iX0) { oB0 = iB0; } else { oB0 = iB1; }	归并门控节点[1]

注： 1. arithmetic.type = .S8, .S16, .S32, .S64, .S128, .U8, .U16, .U32, .U64, .U128, .F16, .F32, .F64, .F128, .F256

2. B.SET2 指令中使用的数据类型 u32 在翻译成机器码时，会被编码。编码值如表 6.2 所示：

表 6.2 各种数据类型的编码

数据类型	U8	U16	U32	U64	U128	S8	S16	S32	S64	S128	F16	F32	F64	F128	F256
编码值	0x03	0x04	0x05	0x06	0x07	0x13	0x14	0x15	0x16	0x17	0x24	0x25	0x26	0x27	0x28

表 6.3 部分浮点加减块指令集

标准指令	标准指令示例	含义	注释
B.ADDVV.type	b.addvv.f32 iB0, iB1, oB0	oB0 = iB0 + iB1	iB0 和 iB1 的数据个数必须相同。
B.ADDVS.type	b.addvs.f32 iB0, iF0, oB0	oB0 = iB0 + iF0	iF0 为 f32 类型的标量数据。
B.ADDVV.M.type	b.addvv.m.f32 iB0, iB1, iM0, oB0	if(iM0(n)) { oB0(n) = iB0(n) + iB1(n) }	iM0 为遮罩数据。
B.ADDVS.M.type	b.addvs.m.f32 iB0, iF0, iM0, oB0	if(iM0(n)) { oB0(n) = iB0(n) + iF0 }	iM0 为遮罩数据。iF0 为 f32 类型的标量数据。
B.SUBVV.type	b.subvv.f32 iB0, iB1, oB0	oB0 = iB0 – iB1	iB0 和 iB1 的数据个数必须相同。

注：arithmetic.type = .F16, .F32, .F64, .F128

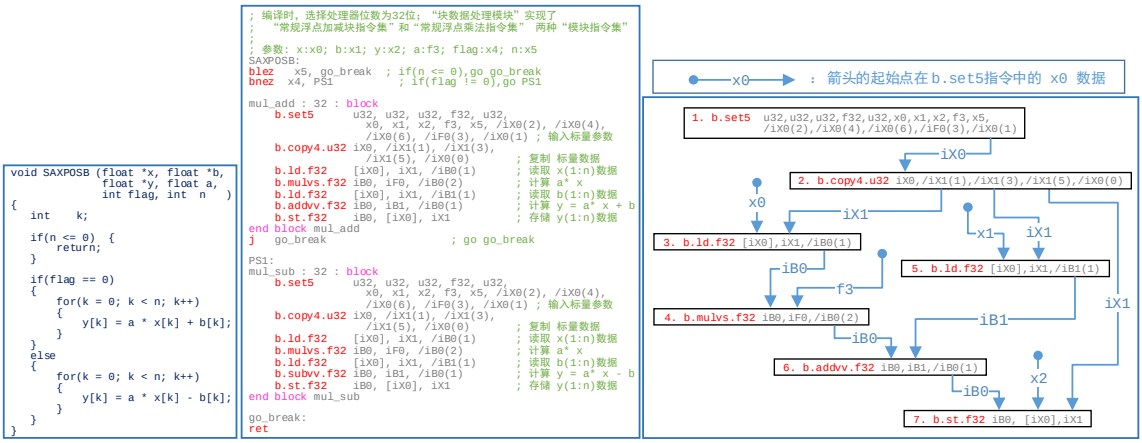
表 6.4 部分浮点乘法指令集

标准指令	标准指令示例	含义	注释
B.MULVV.type	b.mulvv.f32 iB0, iB1, oB0	oB0 = iB0 * iB1	点乘

B.MULVS.type	b.mulvs.f32 iB0, iF0, oB0	oB0 = iB0 * iF0	iF0 为 f32 类型的标量数据。
B.MULVV.M.type	b.mulvv.m.f32 iB0, iB1, iM0, oB0	if(iM0(n)) { oB0(n) = iB0(n) * iB1(n) }	iM0 为遮罩数据。

注：arithmetic.type = .F16, .F32, .F64, .F128

6.3.4 “块数据处理模块”的程序处理模型



(a) C语言函数　　(b) 编译后的代码　　(c) 块指令段mul_add 的执行过程

图 6.4 使用了块指令的函数编译及运行流程。(a)C 语言编写的函数 SAXPOSB。(b)函数 SAXPOSB 编译后的汇编代码。(c)“块执行段”mul_add 的执行过程。（忽略了判断“块数据处理模块”是否能执行此“块执行段”的指令）

之前介绍的标量和向量处理模块，都已被广泛应用，有很多可以参考的实例。但“块数据处理模块”是一种新的模块，很多处理方式都需要探索。如图 6.4 示例，通过对函数的编译，运行流程来简单说明 BZCP 处理器里“块数据处理模块”的处理模型。（“块数据处理模块”翻译后的指令执行过程与此类似，但并不相同）

图 6.4(a)使用的是 C 语言编写的。可以看出，C 语言并不合适描述这种并行运算，需要改进。图 6.4(b)的指令与当前通用的指令有很大不同。分析其中的一条指令：

指令的标准形式	指令的使用形式
b.mulvs.f32 iB0, iF0, oB0	b.mulvs.f32 iB0, iF0, /iB0(2)

指令包含 3 个部分：操作码，输入数据，输出数据。

指令使用时，输出数据形式为：“/iB0(2)”，它由 3 部分组成：1：字符‘/’表示当前指令的一个输出数据；2：“iB0”是后续指令的输入参数标号；3：“(2)”是后续指令相对当前指令的位置为 2。“/iB0(2)”的含义是：此指令的输出数据“oB0”被直接传送到后续第 2 条指令的输入数据“iB0”中。

图 6.4(c)清楚表明：使用块指令的程序模型采用了与数据流计算机[1]一样的数据驱动机制。每条指令，只要所需数据到达，就能独立开始执行。

在将块指令的汇编代码翻译成机器码时，操作码直接翻译成二进制，输入参数被忽略，每个输出数据被转换成使用此数据的目的地。目的地由“相对位置”和“操作数序号”两个字段组成，共占用 16bit。数据形式如下：



相对位置是当前指令和目的地指令之间相隔的距离。比如：当前指令与下一条指令之间的相对位置为 1，与上一条指令的相对位置为 -1。相对位置的编码为 12bit 有符号整数，其值域为 [-2048, 2047]。数字 0 为特殊数据，表示此输出数据没有目的地。如果相对位置超出值域范围，则使用指令（如：B.NOP1.TYPE）中转。

操作数序号是指目的地指令的第几个输入参数。输入参数不区分数据类型。第一个输入参数的序号为 0。操作数序号的编码为 4bit 无符号整数，其值域为 [0, 15]。块指令的输入参数个数不能超过 16 个。

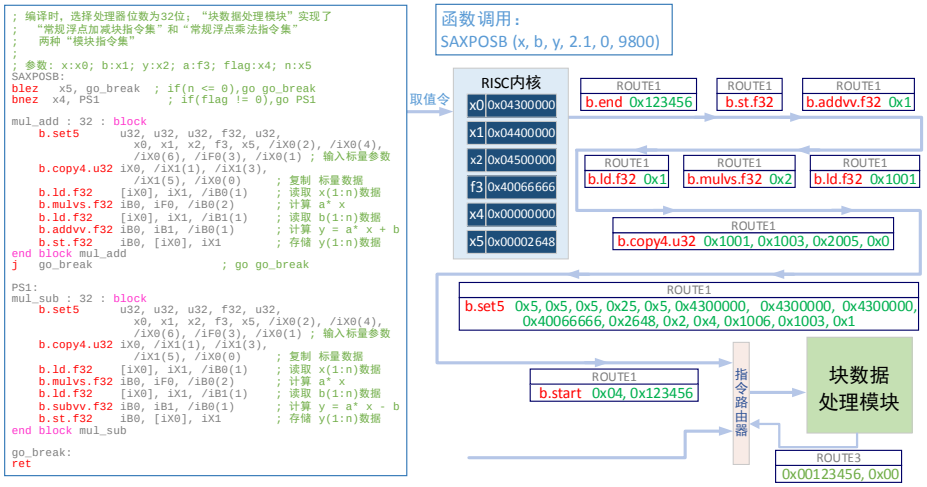


图 6.5 “块数据处理模块”接收运算帧示意图

（忽略了判断“块数据处理模块”是否能执行“块执行段”的条件指令）

图 6.5 所示为“块数据处理模块”接收运算帧的过程。线程运行到“块执行段”内，当不是块指令时，按照正常的指令执行方式处理。遇到块指令时，将指令中对应寄存器的数据取出，封装成运算帧发给“块数据处理模块”。当“块数据处理模块”接收到完整的“块执行段”后，才会开始翻译执行。执行完毕的会返回信息帧给原线程。

6.3.5 “块数据处理模块”的函数

“块执行段”可以实现特定功能，但是不能重用。为了重用代码，我们可以通过使用伪指令的方式来实现“块数据处理模块”的函数定义和使用。#

表 6.5 部分伪块指令集

标准指令	标准指令示例	含义	注释
B.CALL	b.call $ccc, iF0, iM0, oF0, oF1, oM0	调用函数名为 ccc 的函数。iF0，iM0 是输入参数；oF0，oF1，oM0 是输出参数。	实现了函数的调用。其它指令的输出数据通过此指令传到函数中的输入参数。
B.FUNPARM	b.funparm 16, iP0, iP1	将输入数据转到后面 b.call 指令的输入参数中。其中，传入的数据序号将增加第一个参数的值(16)。	函数的输入参数太多，超过操作数序号(15)的范围时，需要使用此指令来传入超过范围的输入数据。此指令只能放在 b.call 或者 b. funparm 指令之前。
B.IN	b.in oF0, oM1	将函数的输入参数转移到相应的输入节点。	此伪指令的参数位置与函数输入参数位置一一对应。输入参数多于 16 个，则连续使用多个此指令。

```
abc :  block fun          ; 函数名为 abc
   in   iB0,iB1,iM0,iF0   ; 函数的输入参数
   out  oB0,oF0           ; 函数的输出参数
begin :
   b.in  /iB0(0),/iB0(1),/iM0(0),/iF0(0)
       块指令
end block fun
```

(a) “块数据处理模块”函数的定义

```
; b.funparm 16,iP0,iP1  ; 当函数xxx的输入参数超过16个，
                        ; 就需要此指令传入参数，
                        ; iP0对应函数xxx中序号为16的输入参数
                        ; iP1对应函数xxx中序号为17的输入参数

b.call  $abc,iB0,iB1,iM0,iF0,/iB0(2),/iF0(5) ;调用函数名为abc的函数
```

(b) “块数据处理模块”函数的使用

图 6.6 “块数据处理模块”函数的定义和使用

与其他编程语言不同，“块数据处理模块”的函数只能以内联的方式嵌入“块执行段”。

6.3.6 块寄存器的兼容性

由于制作工艺，应用场景等不同，不同的“块数据处理模块”内，“块寄存器”的大小会有差异。为了使汇编指令在不同的芯片内都能高效运行，在编译器将程序编译成块指令时，可以通过两种方式实现兼容性：

第一种方式：不管要处理的数据占用多少存储空间，在块指令中都使用一个块数据表示。执行程序，将“块执行段”发送到“块数据处理模块”，在“块数据处理模块”将“块执行段”翻译成本模块的指令时，根据本模块内“块寄存器”的大小，将要处理的数据拆分放入 N 个（N 大于等于 1）“块寄存器”里，再运算。

第二种方式：设“块寄存器”的大小为常数 K。在块指令中，将要处理的数据以常数 K 为大小拆分成多个块数据，然后分别处理每个块数据。执行程序，将“块执行段”发送到“块数据处理模块”，在“块数据处理模块”将“块执行段”翻译成本模块的指令时，使用本模块内“块寄存器”的大小替换常数 K，将每个块数据存放在一个“块寄存器”里，再运算。

这两种方式都是在“块数据处理模块”里，将要处理的数据划分成并保存在 N 个（N 大于等于 1）“块寄存器”里。第一种方式是由“块数据处理模块”自动划分；第二种方式是程序员手动划分。这两种方式适用于不同应用场景，而且它们是可以共存与同一个“块执行段”中。划分后的块数据也许能并行运算，这儿就不展开了。

6.3.7 块指令集的兼容性

针对不同的应用场景，芯片内包含的“复杂运算模块”会有所不同，能够执行的“功能模块指令集”也不同。这将导致某些程序在编译时为了通用性而只使用常规的“功能模块指令集”，不能有效利用芯片资源。

为了解决这个问题，对大量数据处理的功能需求编写三种函数，每个函数适应不同的硬件环境（如果性能要求不高时，可以只写一个函数，然后通过编译器自动完成三种函数的实现）。第一种函数使用“块数据处理模块”中专用的“功能模块指令集”实现（同一功能可以使用不同的“功能模块指令集”实现，能编写多个不同的函数）；第二种函数使用向量指令集实现；第三种函数使用通用的标量指令实现。在程序运行时，根据当前的硬件环境，调用优先级最高的功能实现函数。

6.3.8 块数据处理模块的总结

根据应用场景选择专用芯片时，不仅仅只关心芯片中复杂运算模块的种类，还需要查看频繁使用的复杂运算模块在芯片中的数量。比如：某个应用场景大量数据都在做浮点加的运算，则选择专用芯片时，需要查看芯片中能实现浮点加的复杂运算模块有多少个，只有足够数量，才能缓解运算资源不足的问题。

为了能包容各种不同运算模型，BZCP 处理器中的“块数据处理模块”被设计为以“块寄存器”为数据运算单元的简单计算模型。以其简单，成其博大。

“块数据处理模块”是一个复杂的模块。对于它，本文只是做了一些简单的探讨。还有很多没有描述的地方，很多方面都需要更深入地研究。“块数据处理模块”只是 ZSSC 架构下为 BZCP 处理器设计的一个高速运算模块。也许在 ZSSC 架构下，存在其他结构的模块，能更完美实现大量数据高速运算。

7 服务系统

服务系统完成了之前由软件实现的一些功能。这些功能都是由内核和硬服务器里的服务线程提供的。硬服务器是服务系统中重要的硬件组成部分。硬服务器运行自己固有的程序。每个硬服务器中至少包含一个进程，进程包含一个或者多个线程，但至少有一个线程是为其他进程服务的服务线程。

进程间通信（IPC）是服务系统中非常重要的核心功能，所有的服务都是通过进程间通信（IPC）实现数据交互的。进程间通信（IPC）包含同步进程间通信（Synchronous Inter-process communication，SIPC）和异步进程间通信（Asynchronous Inter-process communication，AIPC）。由于进程只是资源的集合，进程间通信（IPC）其实是在属于不同进程的线程之间传递信息。

7.1 服务线程简介

同步进程间通信（SIPC）参考了客户端-服务器模式（Client–server model）。当线程在“IPC 地址服务器”中注册了至少一个服务端口（servicePort），则此线程成为服务器线程，能接收并处理其他线程发送来的服务帧。其他线程通过关键字查询“IPC 地址服务器”得到服务器线程的进程 ID 和服务端口 ID 后，就能向此进程的端口发送服务帧（忽略操作权限）。服务端口类似函数接口，每个服务端口为客户端线程提供一种服务。

异步进程间通信（AIPC）参考了观察者模式（Observer Pattern）。当线程在“IPC 地址服务器”中注册了至少一个事件主题（evenType），则此线程成为被观察者线程，就能接收并处理其他线程发送来的服务帧。其他线程通过关键字查询“IPC 地址服务器”得到被观察者线程的进程 ID 和事件主题 ID 后，就能向此进程发送绑定的服务帧（忽略操作权限）。当事件主题对应的事件发生变化时，被观察者线程会发送服务帧到关注此事件主题，并符合触发条件的观察者线程。

服务器线程和被观察者线程统称为服务线程。一般一个线程不能同时成为服务器线程和被观察者线程。

一个进程可以包含多个服务器线程，每个服务器线程可以处理多个服务端口。在一个进程内注册的所有服务

端口的 ID 值不能重叠。

一个进程可以包含多个被观察者线程，每个被观察者线程可以处理多个事件主题。在一个进程内注册的所有事件主题的 ID 值不能重叠。

7.2 硬服务器的结构

硬服务器是为系统提供特定服务的微型服务器模块，它的内部结构没有固定形式，在 BZCP 处理器内的结构形式是片上系统（System-on-a-Chip，SOC）。在图 2.1（b）中，“同步 IPC 服务器”，“IPC 地址服务器”，“线程同步子服务器”，“系统信息服务器”等都是硬服务器。

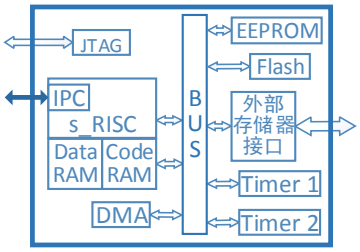


图 7.1　典型硬服务器的结构图

图 7.1 是 SOC 结构的硬服务器。它包含了 RISC 处理器（s_RISC），Flash，ram 和其他外设。

复位后，硬服务器中的 s_RISC 处理器会将 Flash 的程序会拷贝到“Code RAM”中，再运行“Code RAM”中的程序。与内核创建进程的方式不同，硬服务器内的进程由硬服务器内部的 s_RISC 处理器创建。硬服务器会将需要与外部交互的进程注册到“进程服务器”。s_RISC 处理器还会创建至少一个线程，并将需要对外交互的线程注册到“线程服务器”。最后，将需要对外服务的线程注册到“IPC 地址服务器”，成为服务线程。硬服务器只能由在“线程服务器”中注册的线程以进程间通信（IPC）方式与外界交互。

在注册进程的过程中，“进程服务器”会将进程提交申请的地址空间参数发送到与此硬服务器相连的局部存储系统，局部存储系统根据这些参数在存储器中创建进程页表，并为进程分配使用空间。存储空间的大小，由进程提交的参数确定，必须是 0 或者 2^N 字节（N 小于等于硬服务器的地址总线宽度）。当空间大小的参数为 0 时，说明此硬服务器内部的存储空间完全满足程序运行需求，不需要为其分配外部存储空间。

每个硬服务器都可以有自己独特的硬件结构，也可以拥有适合自己的特殊指令集。

7.3 服务指令集 V0.1

服务指令集是一种全新的指令集。它的运行方式与复杂运算指令集相似：内核将指令封装成服务帧后发送到相应的模块中处理。所有的线程（包含服务线程）都是通过使用服务指令来与服务系统中的服务线程交互。

服务指令集属于 ZISA 指令集，也必须实现向下兼容。服务指令有三种类型：第一种是实现系统调用（system call）的指令（后面简称为：系统调用指令）；第二种是通过同步进程间通信（SIPC）实现远程函数调用的指令（后面简称为：同步通信指令）；第三种是通过异步进程间通信（AIPC）实现进程间触发机制的指令（后面简称为：异步通信指令）。

表 7.1 部分系统调用指令

指令	含义
s.creatProcess message	根据参数 message 创建进程。
s.creatThread message	根据参数 message 创建线程。
s.getTime message	获取时间。

表 7.2 部分同步通信指令

指令	含义
s.send dest, servicePort, $\frac{[\text{message}]}{[\text{parameter}]}$	向进程 dest 中的服务端口（servicePort）发送一条同步消息 message。
s.receive source, thread	服务线程接收一条来自进程 source 内线程 thread（或进程 source 的任意线程，或任意进程的任意线程）发送来的同步消息 message。
s.call dest, servicePort, $\frac{[\text{message}]}{[\text{parameter}]}$	向进程 dest 中的服务端口（servicePort）发送一条同步消息 message，并等待应答。

表 7.3 部分异步通信指令

指令	含义
s.add dest, eventType, $\frac{[\text{message}]}{[\text{vid, parameter}]}$	关注进程 dest 的事件主题（eventType），并将其触发 id 设置为 vid。
s.remove dest, eventType	取消关注进程 dest 内的事件主题（eventType）。
s. leisure eventType, n	服务器进程 dest 的事件主题（eventType）可以处理 n 条 add 或者 remove 服务帧。由服务器进程 dest 中的线程发送到异步进程间通信服务器的指令。属于流量控制功能的指令。
s.notify source, thread, $\frac{[\text{message}]}{[\text{vid, information}]}$	被观察者线程发现事件主题（eventType）满足进程 source 里观察者线程 thread 的触发条件，随即发送触发 id 为 vid 的服务帧给观察者线程 thread，并将一些参数放置在 information 里。

系统调用指令参考了远程过程调用（RPC）的概念，为软件提供必要的基础系统服务，通常会被频繁调用。系统调用指令的执行过程与同步通信指令的差不多，都是通过同步进程间通信（SIPC）实现的。系统调用指令与同步通信指令的区别：系统调用指令不需要在指令中设置服务线程所在的进程 ID 和服务端口 ID。处理器芯片上电初始化时就会在服务路由器上设置路由信息，这些路由信息会使系统调用指令封装的服务帧以最短路径直接到达相应的硬服务器。

在同步通信指令中，dest 进程是服务器线程所在进程的进程 ID；source 是客户端线程所在进程的进程 ID（值为 0 表示任意进程）；thread 是客户端线程的线程 ID（值为 0 表示任意线程）；servicePort 是服务线程提供的服务端口 ID；parameter 是服务端口的输入参数。服务器线程在处理完一个 Call 指令的服务帧后，会返回一个包含 N 个（N 大于等于 0）返回值的信息帧给客户端线程。

在异步通信指令中，dest 进程是被观察者线程所在进程的进程 ID；source 是观察者线程所在进程的进程 ID；thread 是观察者线程的线程 ID；eventType 是被观察者线程提供的事件主题 ID，对应一个返回布尔类型的函数；vid 是观察者线程设置的中断编号值，它与当前事件主题 ID 相关联；parameter 是事件主题的输入参数；information 是被观察线程给观察者线程的一些事件信息。当被观察者线程中的事件发生变化，当且仅当事件主题对应的函数在输入 parameter 参数后返回真值（TRUE），被观察者线程才会发送 notify 指令的服务帧给相应的观察者线程。观察者线程接收到服务帧后会进入中断编号为 vid，输入参数为 information 的中断处理程序中。

服务指令中，message 中的 parameter 和 information 是服务线程中，服务端口和事件主题对应函数原型的输入/输出参数（输出参数个数可以大于 1）。按照函数原型中参数的定义，其参数数据依次保存在寄存器中,可以使用[Xn – Xm] (m>n)表示。

一条服务指令的助记符和汇编形式：

```
s.call  process, servicePort, [message]
s.call      x0,         x1,    [ x3 – x9 ]
```

表示：进程 process 的 ID 值为 x0；服务端口 servicePort 的值为 x1；message 有多个参数，依次存放在 x3 至 x9 的寄存器中。

为了软硬件兼容，我们需要对系统调用指令和同步通信指令制定一套类似便携式操作系统接口（Portable Operating System Interface，POSIX）的“系统接口标准”。

当系统调用指令和同步通信指令的操作数太多（可能需要传输包含大量数据的参数），不能将所有的操作数放在通用寄存器时，可以将这些数据放入本线程 A 的内存中，然后在指令的操作数中设置这些数据的起始地址和长度。服务线程接收到此服务帧后，会直接到线程 A 的内存中读出这些数据（具体操作流程参见 7.4 节中的收发大量数据的同步进程间通信流程）。指令返回大量数据时，也可以按照此方式处理。对于异步通信指令，指令的所有操作数都必须能够放在通用寄存器中。

7.4 同步通信指令的处理过程

同步通信指令是通过同步进程间通信（SIPC）完成的。所有的同步通信指令都会被发送到同步进程间通信服务器（SIPC 服务器）。下面将详细介绍其处理过程。

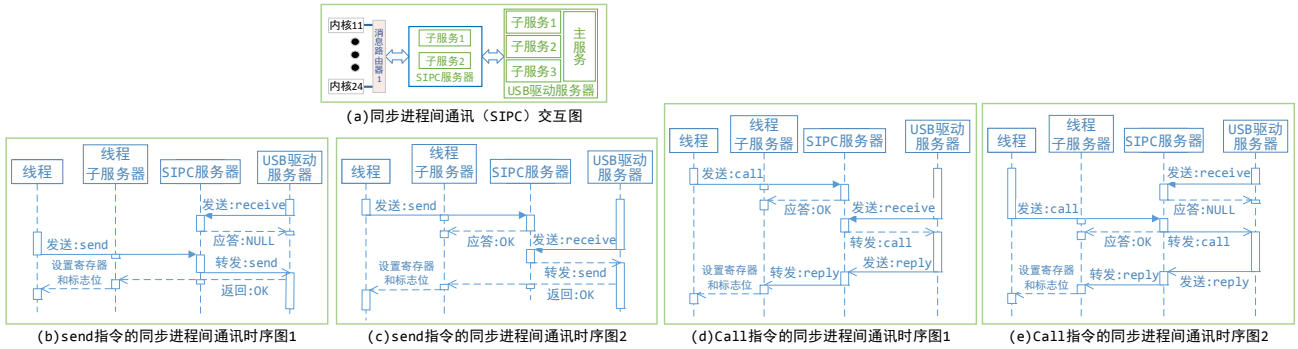


图 7.2　同步进程间通信（SIPC）的流程

图 7.2（c）中，send 指令的运行过程：内核里的线程运行到 send 指令，内核将 send 指令封装成服务帧发送出去，然后线程子服务器将线程设置为“阻塞状态”。服务帧经过“消息路由器”到达“SIPC 服务器”。“SIPC 服务器”判断其目标服务线程不在等待信息的状态，于是暂存此服务帧，并发送给线程子服务器一条“应答：OK”信息帧。当“USB 驱动服务器”中的目标服务线程发送一条 receive 指令的服务帧给“SIPC 服务器”时，“SIPC 服务器”就将暂存的服务帧转发给“USB 驱动服务器”中相应的服务线程。“USB 驱动服务器”接收到正确的服务帧后返回一条“返回：OK”信息帧经过“SIPC 服务器”到线程子服务器。然后，“USB 驱动服务器”中相应的服务线程开始处理此消息。线程子服务器接收到此信息帧后，设置线程中的寄存器和标志寄存器的相应标志位，并将相应线程设置为“运行状态”。

当服务帧从线程发送到“SIPC 服务器”后，“SIPC 服务器”会给线程子服务器一个应答。这个应答可能有以下几种类型：0：OK；1：校验码错误；2：call 和 send 指令使用错误；3：服务器故障（如：服务未注册）；4：操作权限不够；等。

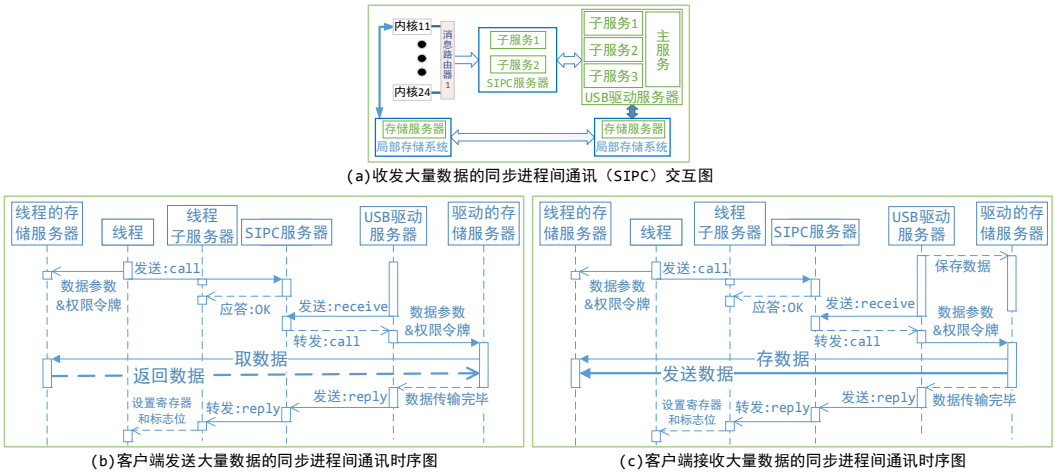


图 7.3　收发大量数据的同步进程间通信（SIPC）流程

收发大量数据的处理流程有一些特别的地方：“驱动的存储服务器”接收到“USB 驱动服务器”发送的消息后，会根据消息中的信息与“线程的存储服务器”交互，由两个存储服务器协同完成大量数据的拷贝操作。

同步通信指令可以实现超时功能，即线程发送到“SIPC 服务器”的消息在一段时间内没有被处理，“SIPC 服务器”根据此消息的超时参数，判断此消息超后，会此消息将被丢弃，同时发送一条超时消息给线程所在的内核。线程子服务器收到超时消息后，修改此线程的运行状态，并设置线程中标志寄存器的相应标志位。

send 指令和 call 指令都是发送消息的指令，但是 send 指令不能有应答，call 指令必须有应答。这两条指令适合不同的应用场景，而且不能相互替换。服务端口限定了是否需要服务器线程应答，即服务端口决定了使用哪种指令（send 或者 call），如果使用了错误的指令，则会收到发送错误的应答。

大量数据的拷贝非常耗费时间。如果只是需要将数据从源进程转移到目标进程，可以使用修改页表方式完成数据转移：存储服务器将数据源所在空间从源进程的虚拟空间修改为目标进程的虚拟空间。通过修改两个进程的页表完成数据转移的操作有一定的限制：需要为数据分配页表中的 N 个页的全部空间，不能因此操作影响两个进程中的其他数据。为了程序在不同页大小等运行环境的兼容性，处理器可以提供为程序分配 N 个完整页的系统调用指令。如果将数据源区域设置为只读属性，则可以由多个进程共享该空间。

我们常常会遇见大量数据多次转移的情况，为了省去中间的拷贝过程，我们可以只取回由数据源的地址，大小，权限等信息构成的数据令牌，然后将令牌传递出去，在需要的时候，由获取此令牌的线程从数据源读取数据。

同步通信指令的服务帧可以由内核中的线程发出，也可以由硬件服务器中的线程发出。

7.5 系统调用指令的处理过程

实现系统调用指令的处理过程与同步进程间通信（SIPC）的处理过程相似，区别在于实现系统调用的指令被封装成服务帧时，会自动添加目的地址，再由“消息路由器”直接发送到目标服务线程，不经过“SIPC 服务器”。当目标服务线程缓存的服务帧太多时，会将服务帧转发到“SIPC 服务器”暂存，之后再取出处理。

7.6 异步通信指令的处理过程

异步通信指令通过异步进程间通信（AIPC）完成的。所有的异步通信指令都会被发送到异步进程间通信服务器（AIPC 服务器）。下面将详细介绍其处理过程。

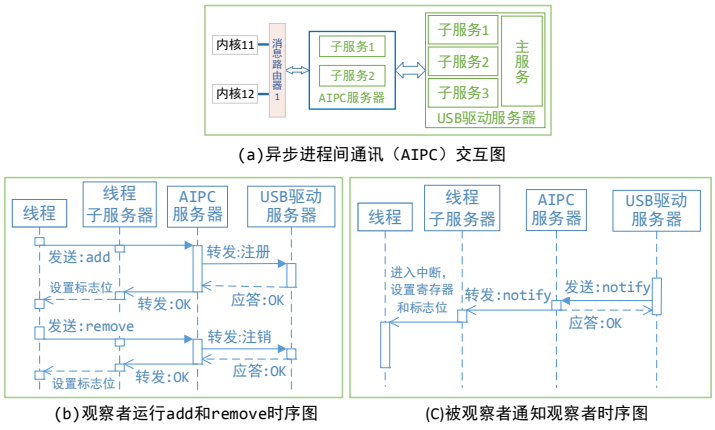


图 7.4　异步进程间通信（AIPC）的流程

图 7.4（b）中，观察者线程运行 add 指令的过程：内核里的线程运行到 add 指令，内核将 add 指令封装成服务帧发送出去，然后线程子服务器将线程设置为“阻塞状态”。服务帧经过“消息路由器”到达“AIPC 服务器”。“AIPC 服务器”先判断“USB 驱动服务器”的服务线程是否在处理 add 或者 remove 服务帧，如果正在处理，则缓存当前服务帧；如果没有处理，则将服务帧转发给“USB 驱动服务器”。“USB 驱动服务器”收到服务帧后将处理主题为 eventType 的被观察者线程转入中断状态（被观察者线程必须能处理此中断），在中断中处理服务帧。被观察者线程中断处理完后，发送给“AIPC 服务器”一条“应答：OK”的消息帧。然后“AIPC 服务器”转发一条“转发：OK”的信息帧给线程子服务器。线程子服务器接收到此信息帧后，将相应观察者线设置为“就绪状态”，并设置线程中标志寄存器的相应标志位。

图 7.4（c）中，被观察者线程触发观察者线程的运行过程：“USB 驱动服务器”产生了事件（如：接收到总线数据），“USB 驱动服务器”内部管理事件主题的被观察者线程就会遍历所有关联线程，将每个关联线程的输入参数都依次带入事件主题对应触发函数中（此操作可以优化）。当某个观察者线程的输入参数使触发函数返回真值（TRUE）时，被观察者线程所在的硬服务器就会将 notify 指令封装成服务帧发送给“AIPC 服务器”。“AIPC 服务器”转发信息给线程子服务器。线程子服务器收到服务帧后，判断观察者线程是否占用精简处理器模块的寄存器组，若线程正在占用，会先保存线程的运行环境；若线程没有占用，则需要先将一个正在运行的线程调度出去，腾出一个寄存器组资源。再将服务帧中的信息（information）写到相应的寄存器中，最后根据服务帧中的 vid 值跳转到相应的中断处理程序中。

“AIPC 服务器”会记录观察者线程和被观察者线程的绑定关系，当被观察者线程由于种种原则发出一条触发服务帧给与其无关联的线程时，“AIPC 服务器”不会转发，并返回给被观察者线程一条“应答：未绑定”的信息帧。

7.7 线程同步服务

BZCP 处理器是多核结构的处理器，同一个进程里的多个线程可能分散在多个内核中运行。BZCP 处理器中处理线程同步的硬服务器是线程同步服务器，它的功能是管理同一进程中多个线程之间的同步操作。

线程同步服务器提供了一组系统调用指令。线程执行这些系统调用指令时，都会被封装后发送到线程同步服

务器中处理。线程间的同步操作都是靠执行这些指令完成的。其具体操作如下：

当线程需要同步操作时，会通过执行一条系统调用指令向线程同步服务器申请一个同步资源。线程同步服务器收到服务帧后为此线程所在进程分配一个资源节点，并将其添加到进程控制块（PCB）的线程同步链表里，然后返回给线程一个资源句柄。线程以此句柄为参数，通过执行不同的系统调用指令完成多线程的同步操作。

当线程通过资源句柄方式实现同步操作时，线程同步服务器只需要不回复接收的服务帧，就会使发送服务帧的线程一直处于“阻塞状态”。当等待超时或者操作完成时，线程同步服务器就会发送应答信息帧给相应的线程。

以系统调用指令为基础，创建一个同步操作的函数库，用于扩展系统调用指令的同步操作，实现各种不同方式的线程同步处理，比如：信号量，互斥锁，读写锁，条件变量等。

BZCP 处理器是多核结构的处理器，有多个缓存模块（cache），会遇到缓存一致性问题（Cache Coherency）[1]。BZCP 处理器对此问题的解决方法是：为同一进程里多个线程共享的数据（只读数据除外）在进程虚拟空间中分配一个特殊的地址段，在这个地址段中不能使用常规读写指令操作这些数据，只能通过系统调用指令进行存取。线程通过服务帧与局部存储系统中的硬服务器交互，实现读写共享数据的操作。这样，读写的数据就不会存放在缓存（cache）里。

BZCP 处理器还支持一条比较交换（Compare and Swap，CAS）的系统调用指令。线程执行比较交换指令时，被内核封装成服务帧后发送到局部存储系统中的硬服务器中，由硬服务器完成比较交换指令的原子操作。

7.8 USB 驱动服务器

BZCP 处理器通过各种总线（bus）与各种外部设备相连。与外部设备通信的总线都由各种总线驱动服务器管理。每个总线驱动模块就是管理着一条或多条总线的硬服务器。

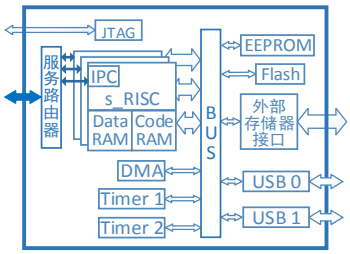


图 7.5　USB 驱动服务器的结构图

图 7.5 是 USB 驱动服务器的结构图。它是一种 SOC 结构，管理着 2 条 USB 总线。USB 驱动服务器包含了多个 RISC 处理器（s_RISC），2 个 USB 主机控制器，Flash，ram 和其他外设模块。

USB 驱动服务器运行两个进程（对应两个 USB 主机控制器），每个进程内包含多个线程。将进程注册到“进程服务器”，为其他模块服务的服务线程注册到“线程服务器”，并根据需要在“IPC 地址服务器”中注册同步服务端口和异步事件主题。

当有新的 USB 设备接入 USB 总线时，USB 驱动服务器会进行设备枚举。服务器获得接入设备的信息后，将这些信息发送给“系统信息服务器”。上层的 USB 设备驱动线程通过查询“系统信息服务器”，获得此 USB 设备的设备信息和服务线程信息，然后通过服务帧与 USB 驱动服务器交互，为应用程序提供驱动服务。

7.9 其他服务器

服务器系统中还有很多其他硬服务器和系统的服务进程，在此就不一一介绍了。

服务器系统中，系统调用指令都是由硬服务器中的服务线程处理。除了一些必须使用硬服务器完成的功能外，其他功能是否使用硬服务器来实现，需要量化研究。一般频繁使用或者安全性要求高的功能可以放在硬服务器中运行。

7.10 服务系统总结

服务系统是 BZCP 处理器的重要组成部分，也是 ZSSC 架构区别于 CISC 和 RISC 的一个重要特征。BZCP 处理器将以软件形式存在的微内核硬件化了。微内核提供的各种服务都被放置在各种硬服务器中，由各种硬服务器提供相应的服务。

由于 BZCP 处理器的服务系统实现了微内核的功能，并且所有服务进程都没有系统进程和用户进程之分。使得运行在 BZCP 处理器的所有进程都没有了系统进程和用户进程的分别。

BZCP 处理器中还存在一些特殊的服务帧和信息帧在处理器中流动。这些帧包是两个硬服务器之间为了相互通信而产生的。这些帧包与用户程序产生的帧包并不一样，它们能完成大量数据的传输功能。为了硬服务器的接口兼容，我们需要对这种特殊的服务帧和信息帧制定一套标准。

8 需要讨论的事项

之前简单描述了属于 ZSSC 架构的 BZCP 处理器，这是一种全新的处理器模型。虽然我们可以有效地利用传统计算机的研究成果，但还是有很多挑战。以下是列举的一些需要讨论的事项。

8.1 多芯片处理器

BZCP 处理器是一个整体，但是为了更好应对各种应用场景，需要配置不同的复杂运算模块。这就需要为不同应用场景单独设计一款芯片，会造成芯片型号过多的问题，也会降低了通用处理器的普适性。

“块数据处理模块”对不同应用场景的数据处理有很大影响，而 BZCP 处理器的其他模块对应用场景的影响不大，并且“块数据处理模块”和其他模块（除了对应的局部存储器）的数据通信量不大。因此可以将“块数据处理模块”单独放置在一颗专用芯片内，其他功能模块放置在一颗通用芯片内，由多颗芯片组成一个完整的 BZCP 处理器。

在组装处理器时，根据需求搭配：用一颗通用芯片配上一颗专用芯片；用一颗通用芯片配上多颗专用芯片；用多颗通用芯片配上一颗专用芯片(每颗通用芯片都是一个完整的 BZCP 处理器)。这样用多颗芯片搭配的 BZCP 处理器，其指令集也会随着专用芯片的不同而变化。

除了运算系统可以扩展专用芯片外，服务系统也需要扩展。服务系统不仅仅能完成资源管理的作用，还可以实现一些运算和数据管理的功能，比如：加密/解密，视频编码/解码，图像信号处理（ISP），数据库管理等等功能。一些不常使用的功能就需要通过扩展芯片的方式来灵活配置。比如专门设计一款数据库服务器的芯片，通过系统总线连接到通用芯片，就能为所有线程提供数据库的加载，增删改查等服务。

多芯片组成的 BZCP 处理器有种种优点，但也增加了一些需要解决的问题：1. 需要制定通用芯片与专用芯片之间的接口标准。想要芯片能够实现管脚到管脚（pin-to-pin）的兼容替换，甚至需要制定芯片管脚标准。2. 需要在专用芯片里增加一个硬服务器。在芯片上电初始化时，通用芯片通过服务帧获取专用芯片的一些信息。在多颗专用芯片的系统中，通用芯片需要为每个专用芯片分配编号，并将每个专用芯片与通用芯片的通信路径等信息与编号关联起来。3. 在多颗专用芯片的系统中，每次需要执行“块执行段”时，都需要分别判断每个专用芯片是否能执行此“块执行段”。如果有多个专用芯片都能执行时，还需要根据这些专用芯片的负载情况，选择轻负载的专业芯片运行此“块执行段”。4. 在多颗通用芯片的处理器中，专用芯片需要记录每个“块执行段”的源路径。等等问题。

8.2 结构相关

本文介绍的 BZCP 处理器都是同构多核结构，但是它也可以被设计成异构多核结构。比如一些运行系统服务线程的内核模块，因为进程的需求确定，所以可以通过修改内核模块的设计，重新排布各种共享模块，使其使用更少的资源，运行更快。这种修改结构的方式有很多缺点，最明显的缺点就是必须将一些特定的线程放置在特定的内核模块中。

8.3 指令集相关

ZISA 指令集不同于 CISC 和 RISC，它是一种新的复杂的指令集。本文只列举了一些 ZISA 指令集的助记符和指令解释，并未完成完整的 ZISA 指令集，也未将指令进行模块化分类。ZISA 指令集的指令编码格式也未介绍。对于一种新的指令集，有很多问题都需要得到解决，以下是列举的几个指令集问题：

1. 为了将来处理器位数提升（升级到 128 位处理器）后，依然能高速运行已有的软件代码，ZISA 指令集需要能够向下兼容。一些新的指令（如服务指令，在 ZISA 指令集中才开始出现）没有历史经验可以借鉴，实现向下兼容并不容易。
2. ZISA 指令集中的块指令不会直接执行，而是被发送到“块数据处理模块”后，翻译成“块数据处理模块”内使用的指令。设计块指令时没有多少可以直接借鉴的资料。
3. ZISA 指令集中的服务指令是全新的指令。设计服务指令时没有多少可以借鉴的指令资料。
4. 执行服务指令时，在指令译码操作后，通常需要从通用寄存器中获取多个操作数（可能会多达数十个操作

数），写回阶段也可能需要将多个结果（可以多达数十个结果）写回通用寄存器。为了完成这些操作，精简处理器模块的结构会被设计的复杂一些。一些其他指令也需要优化（如：执行整数除法指令时，一次性返回商和余数）。

5. 不同芯片的 ZISA 指令集可能不同。在执行用户程序时，需要先判断当前程序能不能在此芯片运行。

8.4 中断和异常

为了简单实现 BZCP 处理器中的精确中断(precise interrupt)，建议指令的每个源寄存器与目标寄存器都不相同，这样即使指令因为种种原因被重复执行了也不会产生错误结果。

对于异常和中断的处理程序，它与唯一线程相关联，是否可以创造一种只有关联线程和对应处理程序能访问的变量。这种变量能在线程与异常中断处理程序之间提供简单，快捷的数据传递，还没有 cache 一致性的问题。

8.5 存储系统相关

本文构造了一个 BZCP 处理器的简单存储系统。实现时，结构和细节都需要修改，完善。

存储系统中缓存的理论依据是程序局部性原理，包括时间局部性和空间局部性[6]。当前通用处理器（如：X86 处理器)的一级缓存(Level 1 Cache)分为一级数据缓存(Data Cache)和一级指令缓存(Instruction Cache)。是否能将数据缓存分成两种类型：一种为“通用数据缓存”；另一种为“栈数据缓存”。因为栈数据通常是按照后进先出（Last-In-First-Out）的方式存取，有空间局部性。

8.6 软件相关

1. 由于 BZCP 处理器支持多输入多输出的服务指令集，因此其应用程序二进制接口（Application Binary Interface，ABI）需要重新定义。
2. 进程 ID 值为 0 有特殊含义，线程 ID 值为 0 有特殊含义。建议保留 0-127 的进程 ID 值和线程 ID 值。
3. 可执行文件（Executable File）需要包含处理器位数，使用的内存大小，使用了那些指令集等信息，如果处理器位数与当前硬件处理器位数不一致，则增加一薄虚拟层来转换。
4. 进程间或者同一进程的多个线程间都可能会发生死锁。应该怎样检测死锁和恢复？
5. 虚拟化技术在计算机系统中应用广泛。BZCP 处理器需要添加一些指令和硬服务器，以实现硬件辅助虚拟化（Hardware-assisted virtualization）的技术。
6. BZCP 处理器是多核系统，它的软件调试是一个不容忽视的难题。

8.7 其他相关

1. 精简处理器模块中通用寄存器的个数从当前的 32 个改为 64 个是否合适？
2. 本文并未描写“块数据处理模块”内部的指令执行过程。
3. BZCP 处理器中的帧包在传输时可能会出现路由器缓存满，帧包数据异常，也可能出现错误的重复传输。需要有能检测出错误，并修正的通信机制。
4. 在 BZCP 处理器里添加权能机制（Capability）。
5. BZCP 处理器中，线程只有“用户态”，没有“内核态”。这对处理器的特权模式，安全性等方面都有较大的影响。而本文并没有描述。
6. 当前芯片的 3D 封装正逐渐普及。为了发挥 3D 封装的优势，BZCP 处理器需要合理安排模块间的相对位置。
7. 本文中，BZCP 处理器的运算系统有三级共享区域（见图 2.1），可以将其压缩为两级共享区域（将复杂标量运算模块和向量处理模块合并到一级共享区域）。

9 总结

本文提出了一种新型通用处理器架构：祝氏共享计算机（ZSSC）。它包含一种新的处理器设计思想。在此设计思想下，构建了一个简单的新型通用处理器模型 BZCP。BZCP 处理器属于冯•诺依曼架构的同构多核的处理器。BZCP 处理器模型尚不完整，并且它不是 ZSSC 架构下唯一的实现方式，更不是最好的实现方式。

BZCP 处理器中有三种指令集：一种是用户程序能够使用的指令集（ZISA 指令集）；一种是“块数据处理模块”中使用的指令集；一种是硬服务器内部使用的指令集（不同的硬服务器，内部的指令集可能也不同）。其中，ZISA 指令集是一种复杂指令集，是用户和系统程序唯一能使用的指令集，其他两种指令集都对系统和用户软件透明。本文只是简单介绍了 ZISA 指令集中的一部分指令，及其运行过程的模拟。

BZCP 处理器可以分为三个部分：一个是多个内核组成的内核区域；一个是多个复杂运算模块组成的运算区域；一个是多个硬服务器组成的服务器区域。这三个部分通过通信网络相连接。通信网络只传输数据量小的运算帧和控制帧，大量的数据迁移都是在存储系统内流通的。因此，通信网络带宽不需要太大。

BZCP 处理器的复杂运算模块分为三层，这三层是递进关系的。第一层是复杂标量运算，运算复杂标量的运算帧中包含运算方式和运算数据，模块一次执行一个运算帧；第二层是向量运算，运算向量的运算帧中只包含运算方式，运算的向量数据在向量处理模块的“寄存器簇”中，模块一次执行一个运算帧；第三层是大量数据高速运算，高速运算大量数据的运算帧中只包含运算方式，运算的数据在处理模块的本地存储器中，处理模块接收到多个运算帧组成的“块执行段”后才开始执行（为什么处理大量数据高速运算的模块要接收多个运算指令后才开始执行？因为在这种模块中，大量数据的移动在时间和能耗上都是不可忽略的。只有接收了多条指令，才可以确定数据的移动轨迹，尽可能将数据直接送到需要它的地方，减少中间缓存环节）。

BZCP 处理器中的“块数据处理模块”是一个比较完整的处理器，有自己的指令集，控制单元，运算单元，寄存器，代码和线程。它提供了一种简化的大规模数据运算模型，这个模型对于上层的软件设计非常友好。

BZCP 处理器中运算模块和服务模块的共享密度（被越多模块共享，共享密度越大）是由需求确定。在某种应用场景中，如果对 sin 函数的使用较多，就让两个内核共享一个 sin 函数的运算模块；如果非常频繁，可以将 sin 函数的运算模块移入内核，放置在单/双精度浮点运算模块的位置。如果对双精度浮点运算使用较少，可以由多个内核共享一个双精度浮点运算模块。如果对“块数据处理模块”中某种复杂运算模块使用比较频繁，则可以在“块数据处理模块”中放置多个这种复杂运算模块。这是一种时间和空间的权衡。BZCP 处理器通过共享的方式即支持用时间换空间，也支持用空间换时间。

处理器是用来运行指令的，从指令的角度，BZCP 处理器可以看成两个同心圆的结构。所有指令从四面八方通过外圆的边界同时进入多个内核模块，其中大部分简单的指令在外环中就被执行了，少部分复杂的指令则进入内环执行。指令执行后就如同被吞噬一样消失了。指令源源不断的产生，然后被同心圆吞噬。BZCP 处理器中的服务系统就是配合着外环的内核模块，加快对简单指令的吞噬速度。让多个进程同时运行测试指令， BZCP 处理器吞噬所需的时间越短，则说明其运算速度越快。所以，BZCP 处理器不追求局部区域的吞噬速度，而是追求整个同心圆的整体吞噬速度。

BZCP 处理器的结构天然覆盖了针对应用领域做优化的处理器架构（DSA）。因为针对应用领域做优化的模块都可以被改造成一种复杂运算模块放入“块数据处理模块”里，相应的控制、运算指令可以创建一种“功能模块指令集”，放入“功能模块指令集标准”中。

BZCP 处理器的服务系统是通用处理器内一种新的功能系统，它不止完成了各种硬件资源的管理服务，还能通过各种数据处理的微型服务器完成各种工作，包含固定算法的算力服务器和大规模数据检索的数据服务器。这是一个充满了极大想象空间的系统，引人入胜。

我在处理器设计的道路上推开了一扇通往新世界的大门，依稀能看见门后引人入胜的景色，但限于个人的能力，只能止步于此。芯片还是那个芯片，功能还是那些功能，但是指导思想变了，架构改了，很多地方也跟着改变。其中既有挑战，也有希望。相信美好的事会发生！

参 考 文 献

[1]张晨曦，王志英. 计算机系统结构教程[M].第 2 版. 北京：清华大学出版社，2018：22，19，385-399，304-312

[2][日]安藤桐. 支撑处理器的技术：永无止境地追求速度的世界[M].李剑译. 北京：电子工业出版社，2012：69-70

[3] John L Hennessy，David A Patterson. A New Golden Age for Computer Architecture[J]. ACM，2019,62(2)：48-60

[4]陈海波，夏虞斌. 现代操作系统：原理与实现[M]. 北京：机械工业出版社，2020：24-25

[5][美]William Stallings. 操作系统：精髓与设计原理(第八版)[M].陈向群，陈渝. 北京：电子工业出版社，2017

[6][美]John L Hennessy，David A Patterson. 计算机体系结构：量化研究方法(第 5 版)[M].贾洪峰. 北京：人民邮电出版社，2013：53-99，72

[7] David A Patterson，Andrew Waterman. The RISC-V Reader: An Open Architecture Atlas(中文版)[M]. Strawberry Canyon，2017

经 历 与 致 谢

2002 年左右，我在学校图书馆阅览一本介绍计算机系统结构的书（书名忘了，只记得是清华大学出版社出版的），翻到数据流计算机的章节，阅读时突然灵光一闪，想着通过共享运算的方式实现多核处理器模型。2006 年开始尝试着使用 FPGA 实现一个简单的八核处理器，并于 2007 年完成。2019 年 4 月构建了“块数据处理模块”，之前一直将其构想为类 GPU 模块。2019 年 8 月底开始意识到需要添加服务系统，此后逐渐完善。

首先，我要非常感谢许勇教授。没有在他领导下的桂林电子科技大学汽车电子研究室下学习和实践，就没有当初那个用 FPGA 实现的简单多核模型，也就没有后来这么多年持续不断的改进，不会有 ZSSC 架构和 BZCP 处理器，更不会有这篇文章。

其次，在写作过程中，就遇见的问题请教过一些网友，得到了他们的帮助。

本文中包含的很多知识都是在书和网络中汲取的，没有一一标注，望见谅。

作者简介：



祝林（1982- ），男，重庆市人。E-mail：zhu1982lin@126.com

ZHU Lin， born in 1982。



翟冬青（1972- ），男，河北省石家庄市人。工信部、民航局国家级评委专家，研究方向：大型客机电传系统计算与控制。E-mail：zhilu108@126.com

ZHAI Dong-qing，born in 1972，Expert from the National Evaluation Committee of the Ministry of Industry and Information Technology and the Civil Aviation Administration, research direction: Calculation and Control of Large Passenger Aircraft Flyby Systems.